



A BRIEF INTRODUCTION TO NP-COMPLETENESS

DATA STRUCTURES AND ALGORITHMS 2
MARK FLORYAN AND AARON BLOOMFIELD

ROADMAP: GOALS FOR THIS DECK

Reductions help us establish one problem is at least as hard as another. Can we find the “hardest problems”

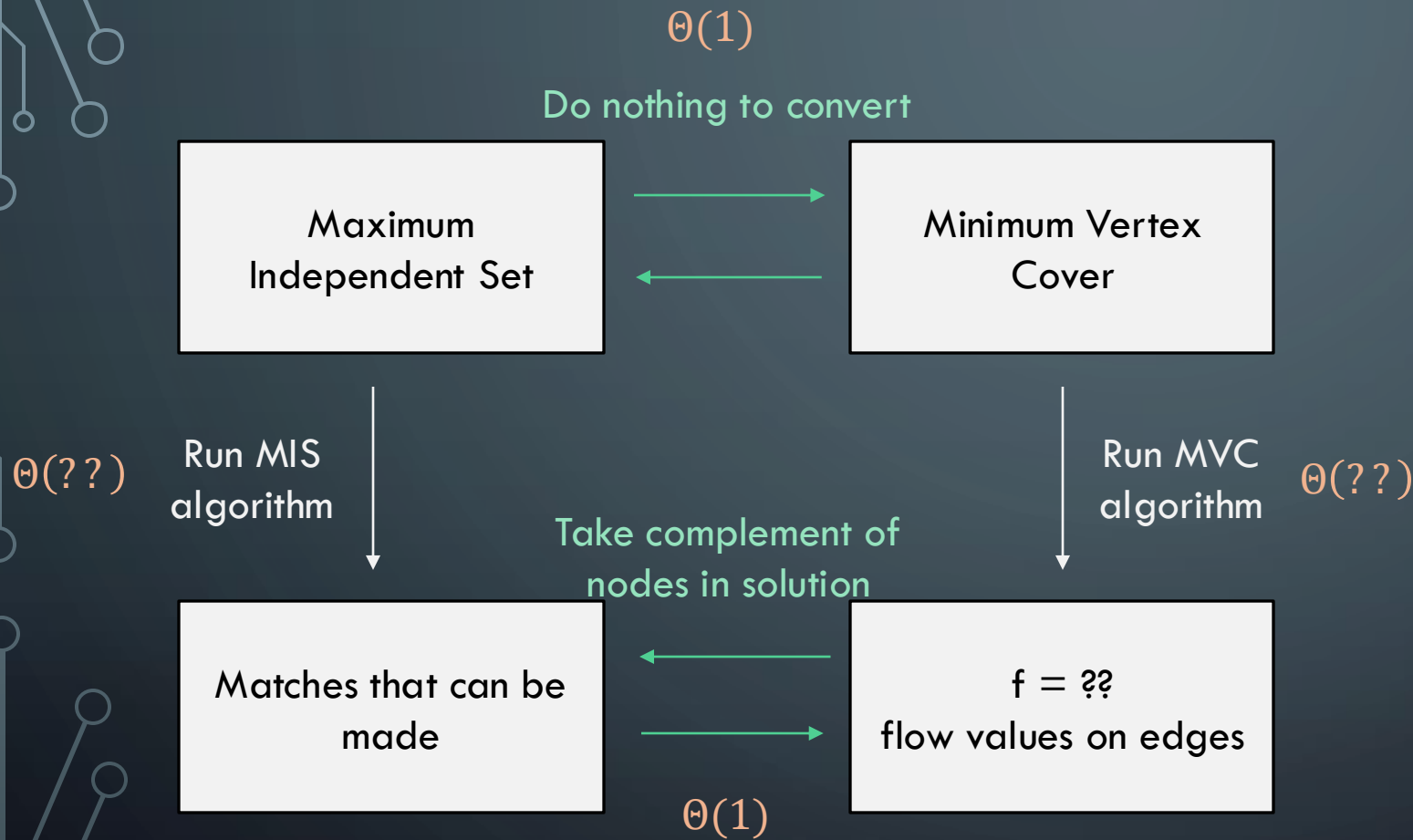
What do these hard problems look like and how do we know that they are the “hardest” in some set of problems.

Along the way: A brief introduction to NP-Complete problems.

The background is a dark blue gradient with a large, faint, light blue circle in the center. In the four corners, there are white line-art illustrations of circuit boards or neural network connections, featuring lines and small circles.

REDUCTION REMINDERS

REMINDER: REDUCTIONS



Reduction Facts:

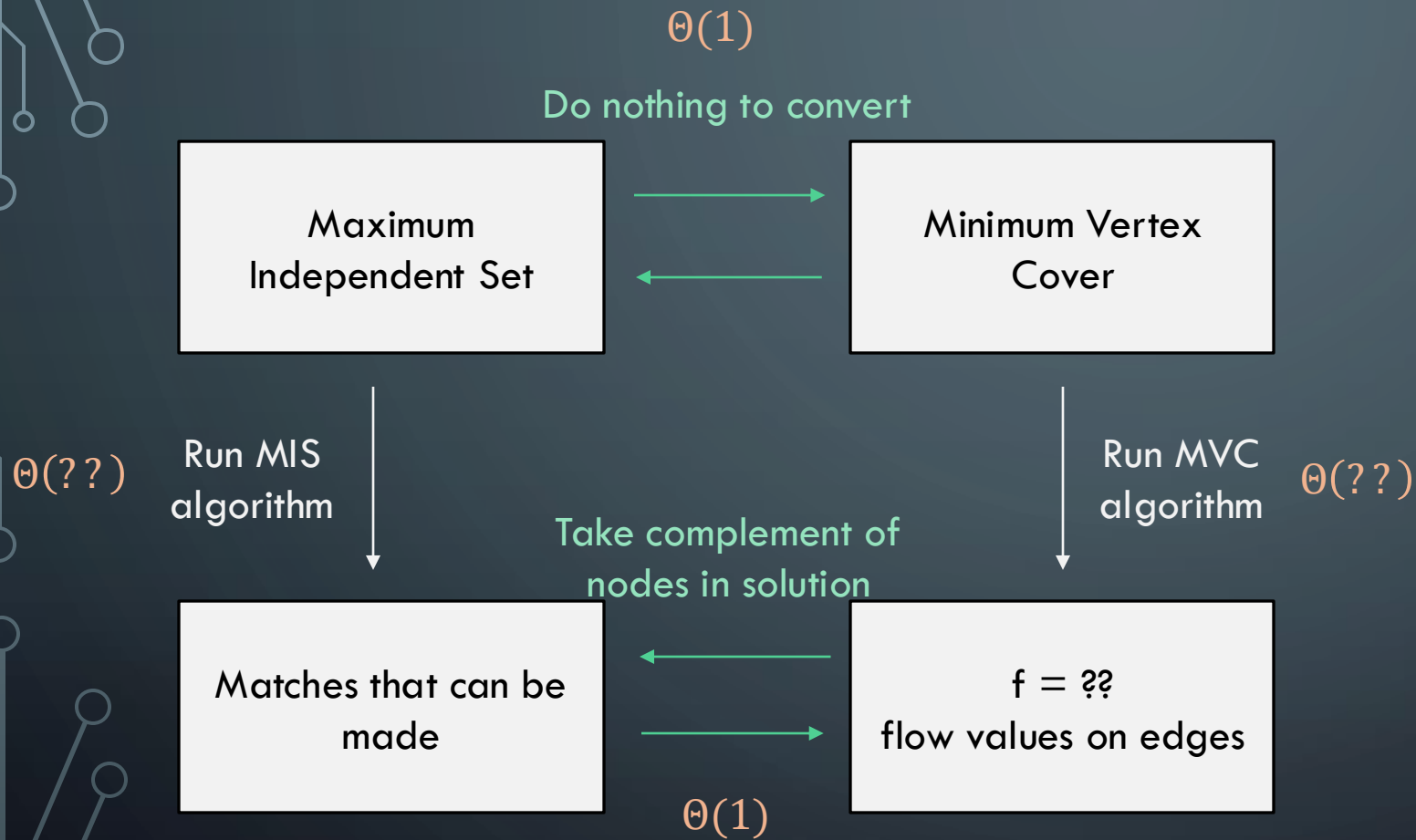
Written as $A \leq_{f(n)} B$ where $f(n)$ is complexity of conversion

Conversion ($f(n)$) needs to be faster than the algorithm for A or B

Establishes that B is at least as hard as A . Difficulty is equal if reduction exists in both directions

Can be used to establish which problems are the **harder than huge numbers of other problems?**

REMINDER: REDUCTIONS



Reduction Facts:

Written as $A \leq_{f(n)} B$ where $f(n)$ is complexity of conversion

Conversion ($f(n)$) needs to be faster than the algorithm for A or B

Establishes that B is at least as hard as A . Difficulty is equal if reduction exists in both directions

Can be used to establish which problems are the **harder than huge numbers of other problems?**

So...can we find the hardest problems that computers need to solve?

COMPLEXITY CLASSES

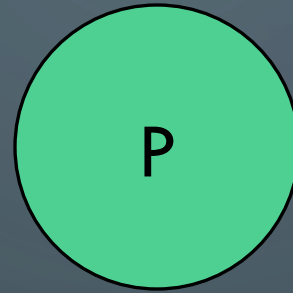
FINDING THE HARDEST PROBLEMS

Goal: We want to identify what some of the hardest problems for computers to solve. We can do this by relating the difficulty of problems through reduction!

Complexity Class: A set (group) of problems that all have the same (or similar) difficulty to solve (given some definition of “same”).

THE CLASS P

Important: *P* is a set of
problems (not solutions,
not algorithms)



Example problems in this set include:

Sorting a list of numbers

Inserting into a binary tree

Computing the average of a list of numbers

Printing "hello world"

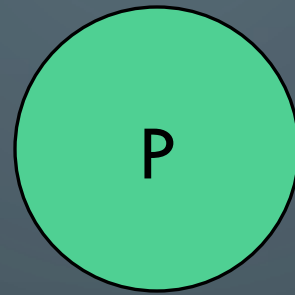
Find() in a hash table

...and many more

The class P is the set of all
problems that can be solved
by a computer in time $O(n^c)$
such that $c \in \mathcal{N}$

THE CLASS P

*Essentially: P is a set of
problems we can solve
quickly*

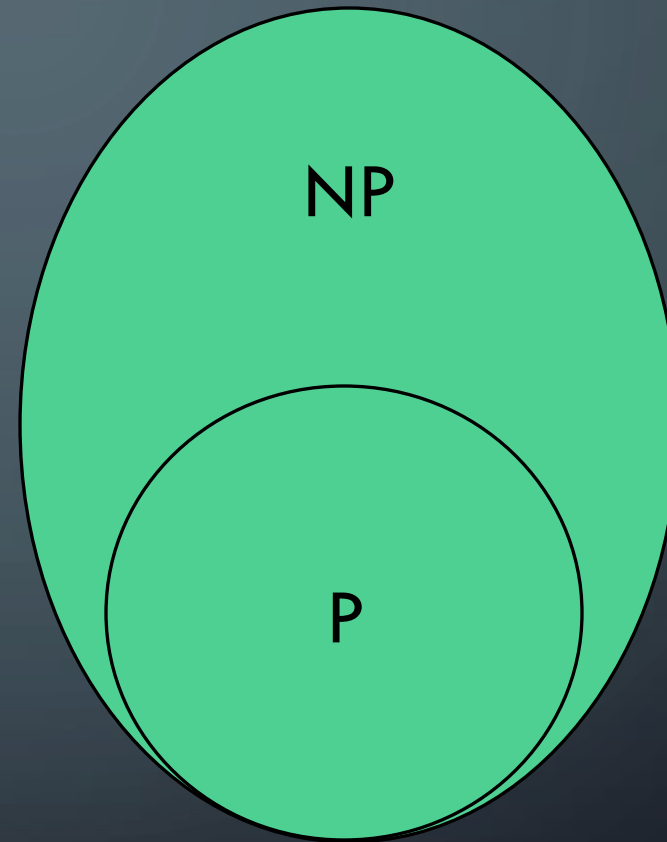


NP

NP :

NP is the set of problems for which solutions can be *verified as correct in $\Theta(n^c)$* (the difficulty of finding solutions varies)

You will see the formal definition and talk about this complexity class in more detail in DMT2!



Hard Problems

Easy Problems

For now, just trust us that $P \subseteq NP$

NP: QUICK VERIFICATION

What does it mean to “verify quickly”?
Consider the following problem?

```
isComposite(int x): Return true if and only if x is composite (not prime)
```

`isComposite(282475249) = ?`

NP: QUICK VERIFICATION

`isComposite(int x)`: Return true if and only if x is composite (not prime)

`isComposite(282475249) = ?`

Solve:

Figure out if `282475249` is composite with no additional info. Pretty hard to do (there are algorithms for this FYI)!

Verify:

Given evidence that `282475249` is composite, verify if evidence is accurate.

Evidence: `7`

To verify, simply check:

`282475249 / 7 = 40353607` (VERIFIED!)

NP: QUICK VERIFICATION

`isComposite(int x)`: Return true if and only if x is composite (not prime)

`isComposite(282475249) = ?`

Since verifying solutions is polynomial time (in this case, $\Theta(1)$), this problem is in the set NP

Verify:

Given evidence that 282475249 is composite, verify if evidence is accurate.

Evidence: 7

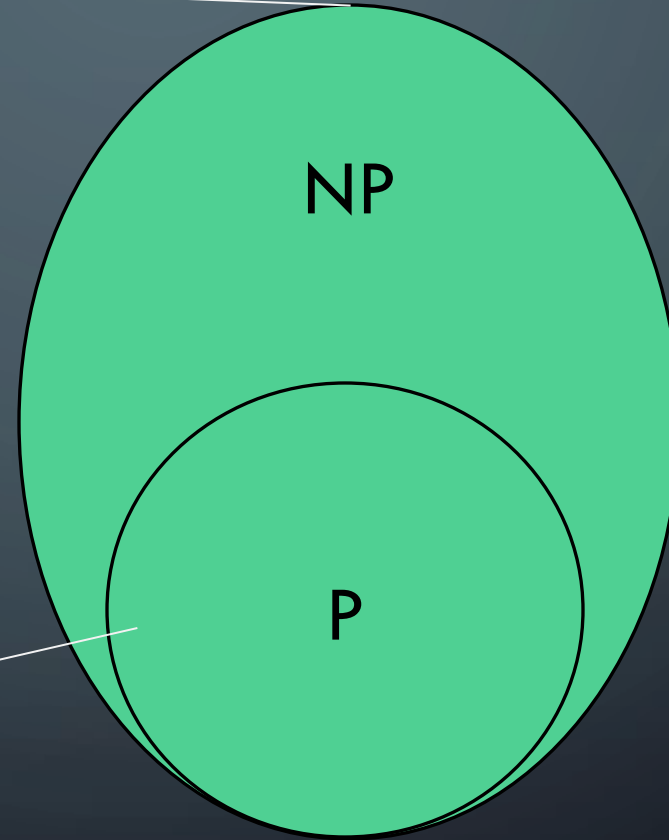
To verify, simply check:

282475249 / 7 = 40353607 (VERIFIED!)

BACK TO OUR DIAGRAM

What is the problem that sits at the very top here? For our purposes, this is the “hardest” problem (hardest in these groups we are choosing to study)

Almost every algorithm we have studied has a polynomial time algorithm and is in P



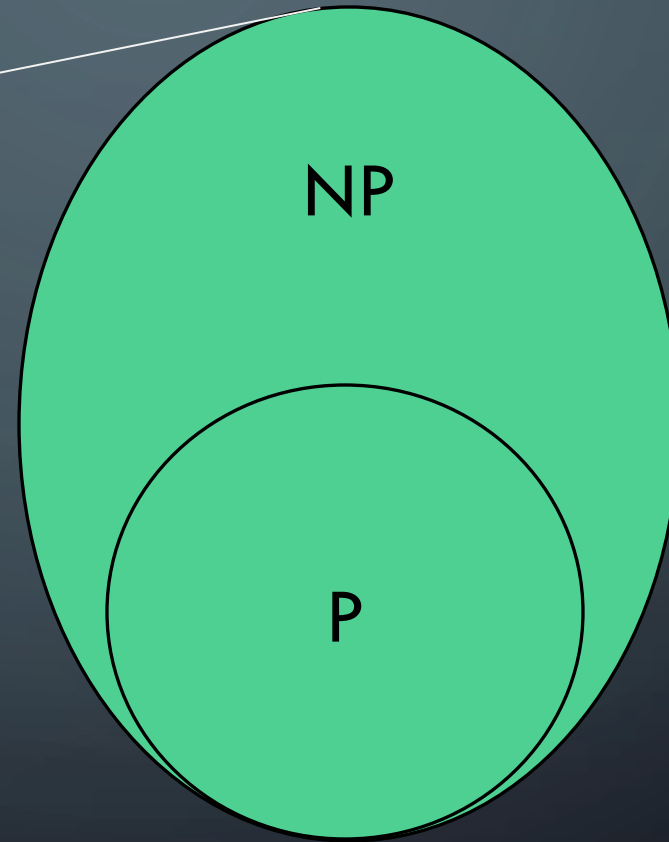
Hard Problems

Easy Problems

NP-HARD

*Suppose we have find the
hardest problem in NP*

*NP-Hard problems are defined to be
all problems that are this hard OR
harder.*

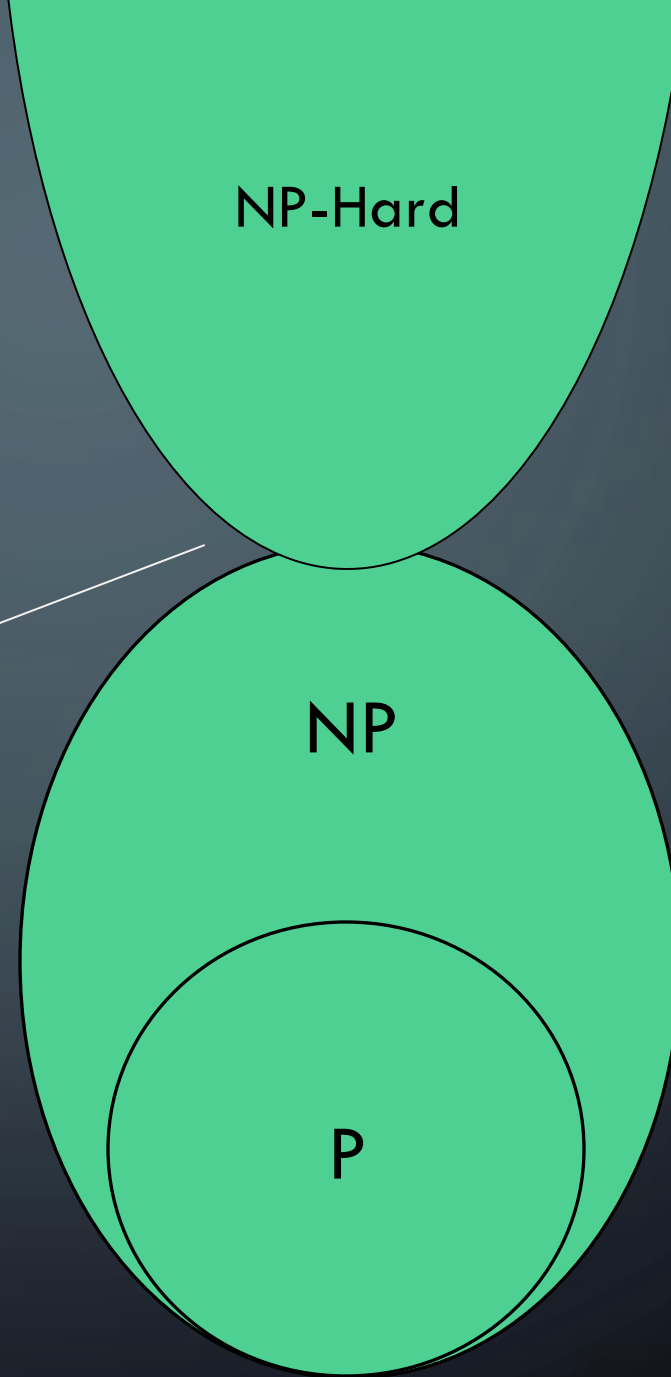


Hard Problems

Easy Problems

Goes up to indefinite difficulty.

Note that NP-Hard and NP intersect here. Problems in this intersection are the hardest problems in NP



NP-Hard

NP

P

NP-HARD

Hard Problems

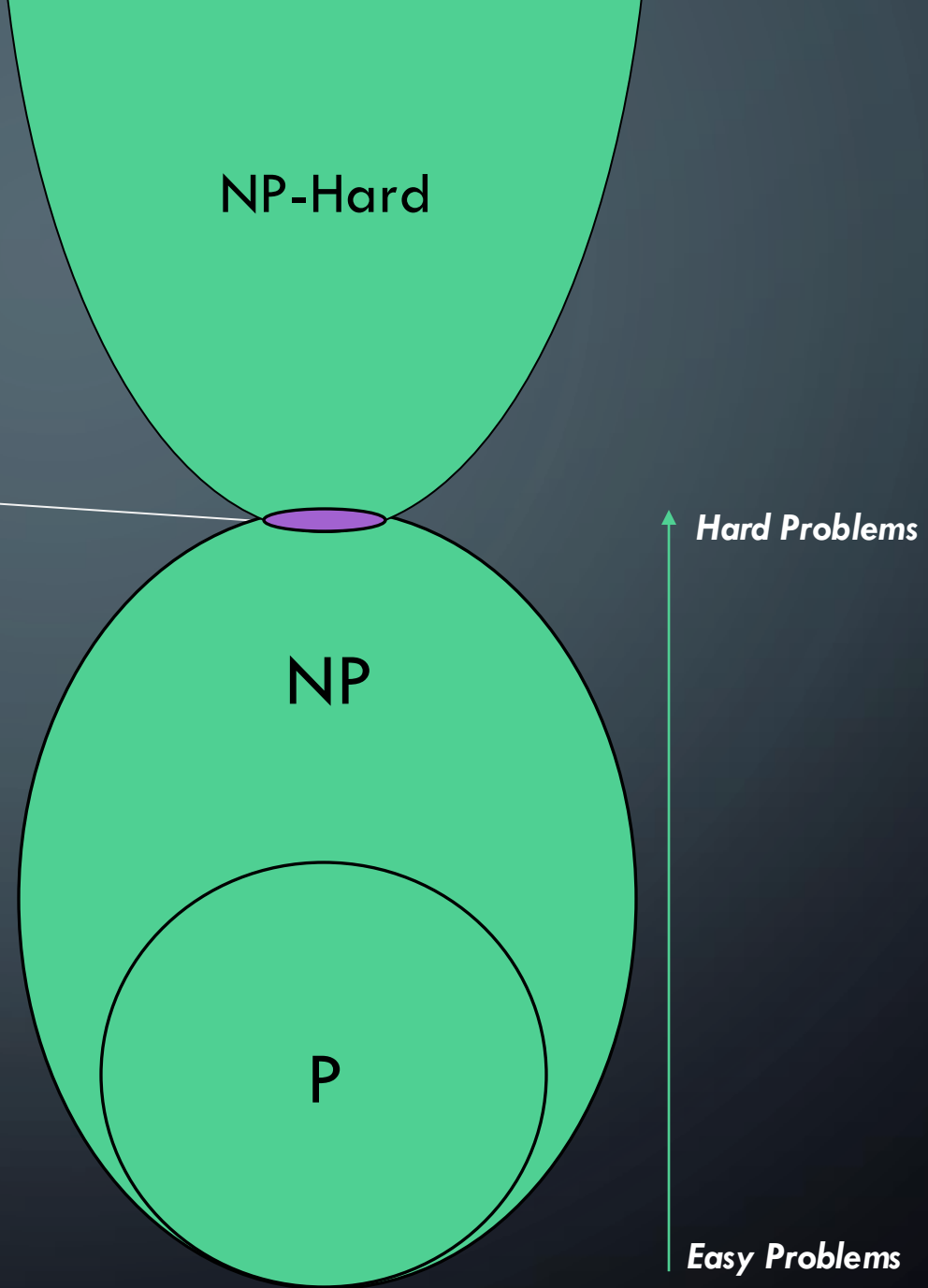
Easy Problems

NP-COMPLETE

This section (purple) is the set of NP-Complete problems. The hardest problems in NP

Definition: A problem is **NP-Complete** if and only if the problem:

1. Is in NP
2. Is NP-Hard

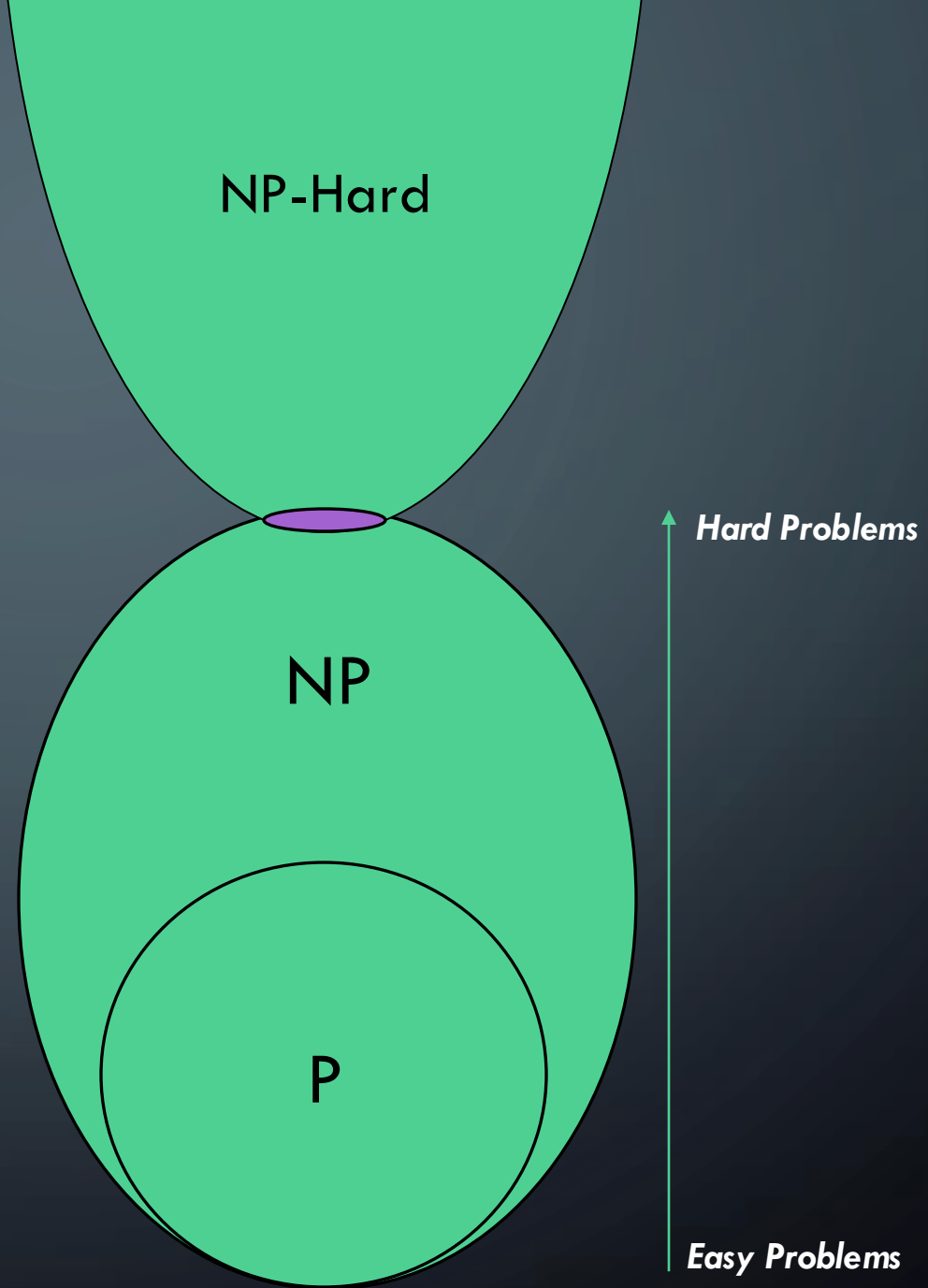


NP-HARD

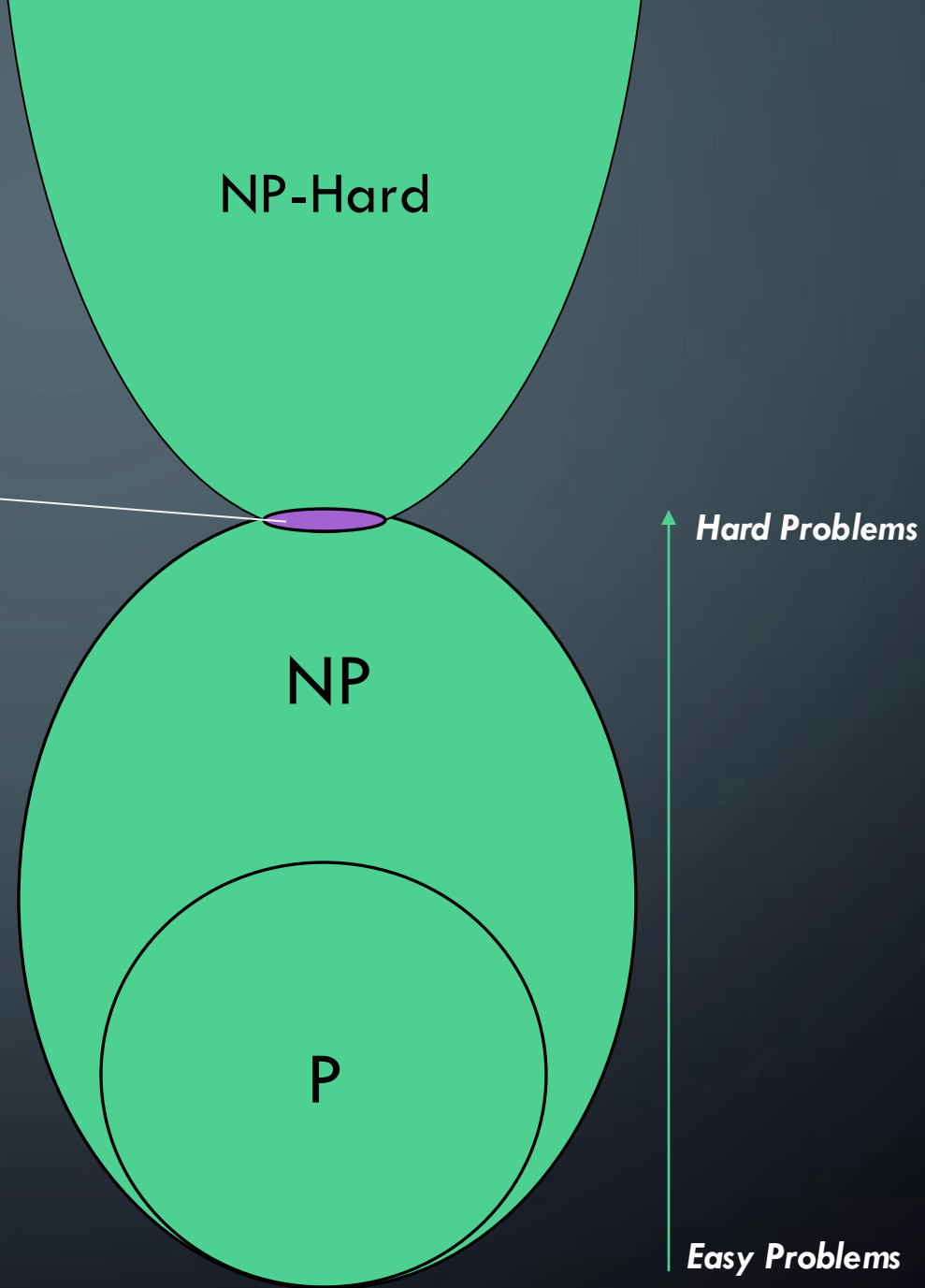
A different definition of NP-Hard

Definition: A problem A is **NP-Hard** if and only if $\forall B \in NP, B \leq_p A$

$B \leq_p A$ means that problem A is harder than problem B , shown through a reduction, which we will see in a moment.



Remember that goal is to find a really hard problem in this NP-Complete region. But what is that problem?



SATISFIABILITY

SATISFIABILITY (SAT)

Satisfiability (SAT) Problem: Given a **boolean formula**, is it possible to assign the variables true false values such that the **entire expression evaluates to “true”**

Example 1:

$$x_1 \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3)$$

True: Set all variables to *True*

Example 2:

$$x_1 \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$$

False: All assignment *Fail*

SATISFIABILITY (SAT)

$$x_1 \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3)$$

...but imagine this formula could be VERY long!

Solve:

Very difficult! Only known algorithms are $\Omega(2^c)$

Verify:

Evidence: $x_1 = T, x_2 = T, x_3 = T$

Plug into formula and check solution.
Algorithm is polynomial time!

Claim: SAT is NP-Complete

...but how do we prove
it? Need to show that
every other problem
down here somewhere

Can be reduced to SAT:

$$A \leq SAT$$

SAT

NP-Hard

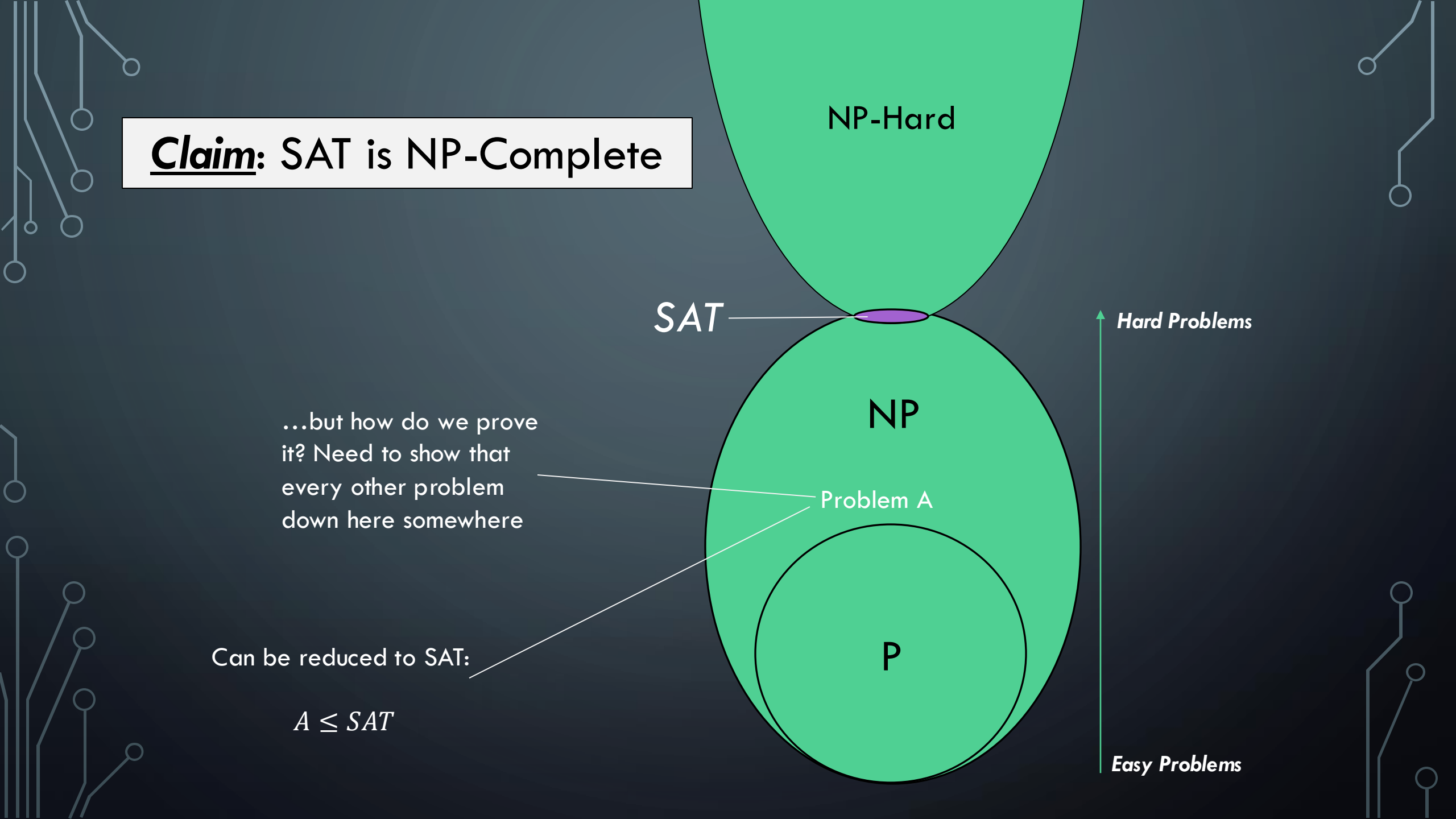
NP

Problem A

P

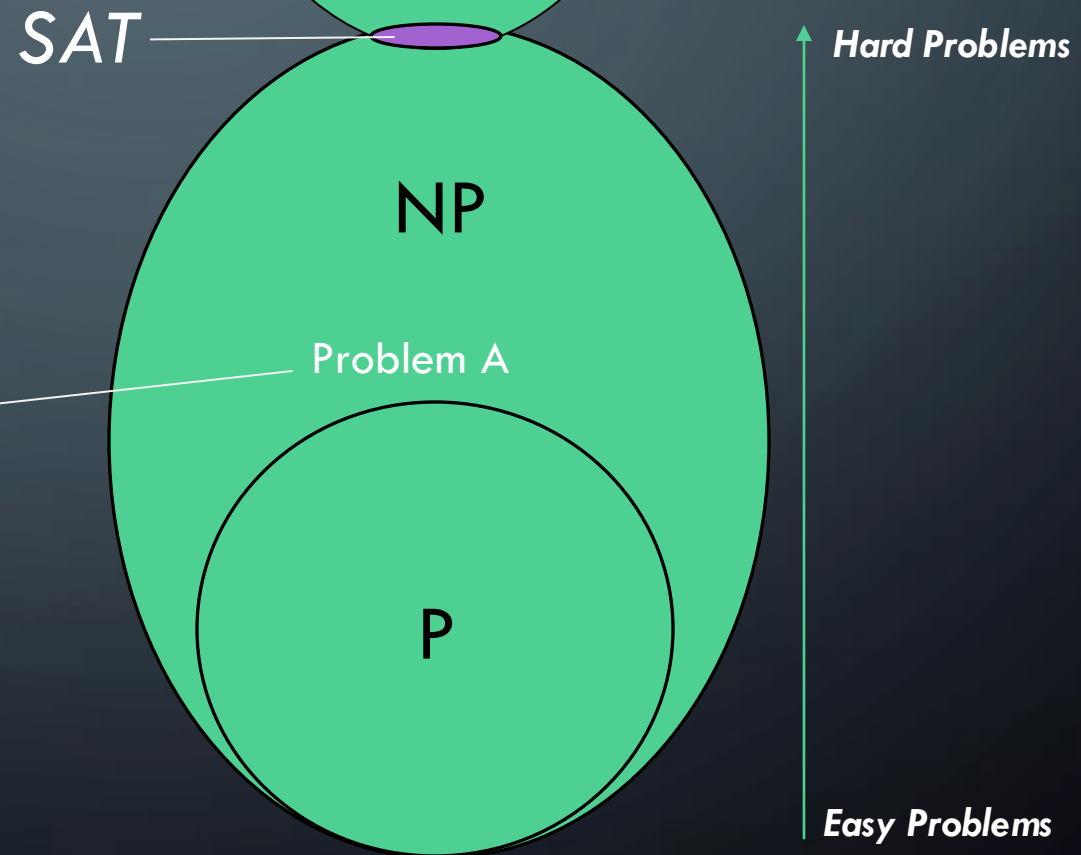
Hard Problems

Easy Problems



Claim: SAT is NP-Complete

What do we know about any generic problem A: It has a computer program that solves it!



COMPUTER PROGRAMS!

QUICK ROADMAP

All computer programs can be represented with
computational models

A **computational model** is a simple reduced version of the program but fully describes how to solve the problem.

There is a reduction between a **computational model's** description (“code”) and an equivalent SAT expression!

Disclaimer!

Everything in this section is going to be a bit vague. It is A LOT of material and we don't have time to go over it thoroughly. This will give you a rough idea of how these things work and you will see it in much more detail in DMT2!

SIMPLE PROGRAM EXAMPLE

Consider the following problem:

Given a **string** input: Is this **string** a valid UVa Computing ID?

Format:

1. Starts with two or three letters (L)
2. Optional digit (D)
3. 1-3 more letters IF there was a digit

Valid Formats:

LL, LLL, LLDL, LLDLL, LLLDLL, LLDLLL, LLLDLLL

PYTHON CODE

```
def check_uva_userid1(what):
    chars = list(what.lower())

    # check first character (must be a letter)
    if len(chars) == 0: return False
    if not chars[0].isalpha(): return False
    chars.pop(0)

    # check second character (must be a letter)
    if len(chars) == 0: return False
    if not chars[0].isalpha(): return False
    chars.pop(0)

    # return true if of the form ll
    if len(chars) == 0: return True

    # check optional 3rd letter
    if chars[0].isalpha():
        chars.pop(0)
        # return true if of the form lll
        if len(chars) == 0: return True

    # check digit
    if len(chars) == 0: return False
    if not chars[0].isdigit(): return False
    chars.pop(0)

    # check first letter after the digit
    if len(chars) == 0: return False
    if not chars[0].isalpha(): return False
    chars.pop(0)
```

```
# return true if of the form lldl or lldl
if len(chars) == 0: return True
# check second letter after the digit
if not chars[0].isalpha(): return False
chars.pop(0)

# return true if of the form lldll or lldll
if len(chars) == 0: return True
# check third letter after the digit
if not chars[0].isalpha(): return False
chars.pop(0)

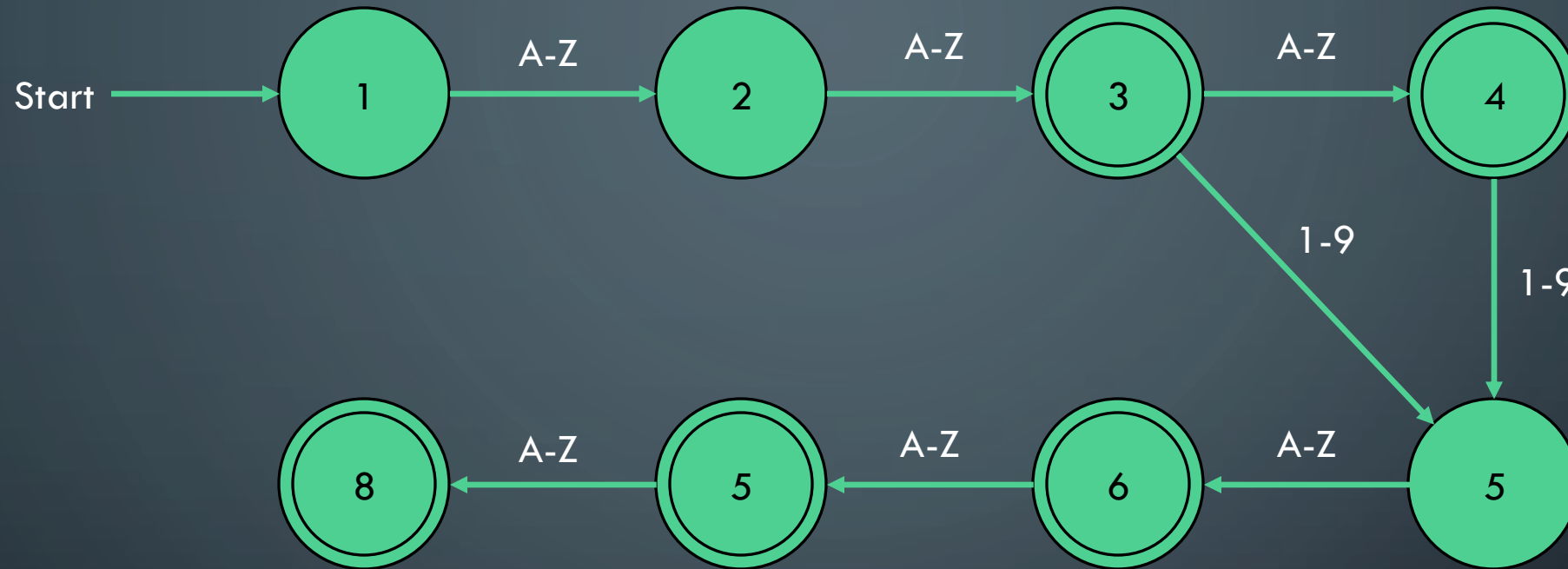
# return true if of the form lldlll or lldlll
if len(chars) == 0: return True

# if there is more input, then it's not a valid userid
return False
```

```
print(check_uva_userid1(input()))
```

Main Point: This code works! It uses a bunch of if-statements and is a working computer program for this problem. Great!

A DIFFERENT WAY OF "CODING" THIS...



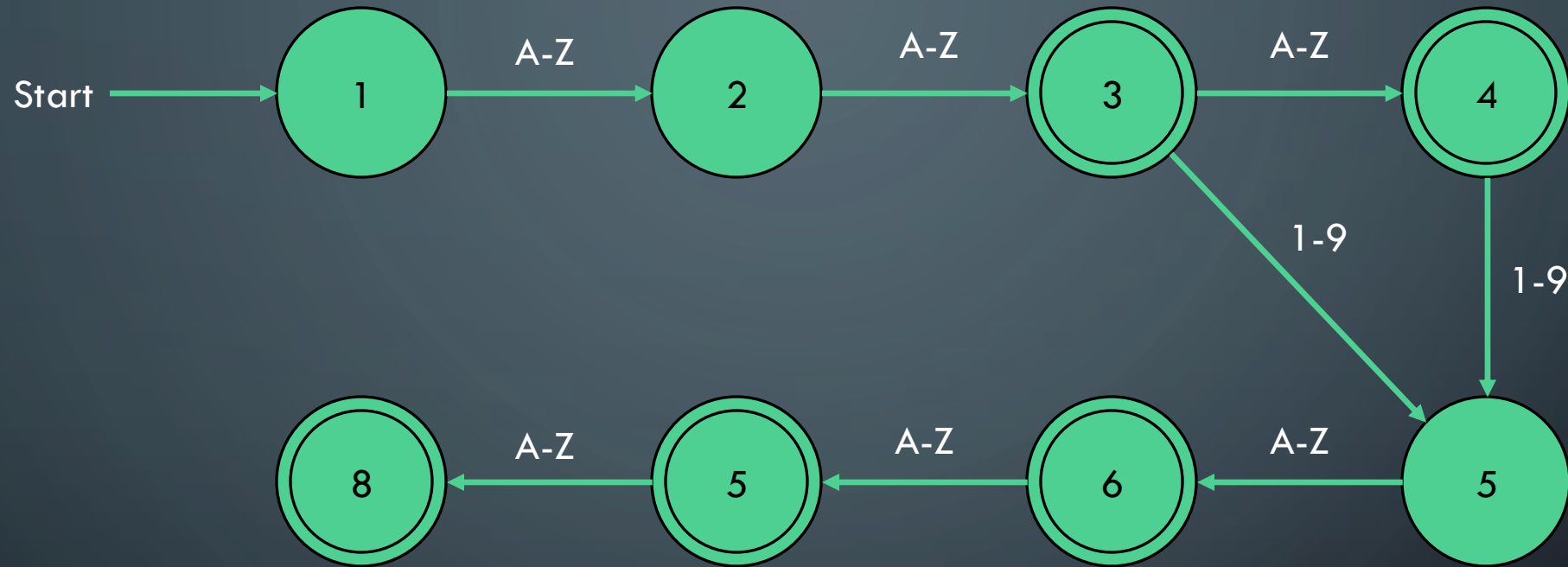
Procedure:

```
Start at the start state
For each symbol of the computing id:
    Go to the next state as per transition labels
If finished and state your end in is accepting (double circles):
    Return True (Valid ID)
Else:
    Return False (Not Valid ID)
```

Try it out:

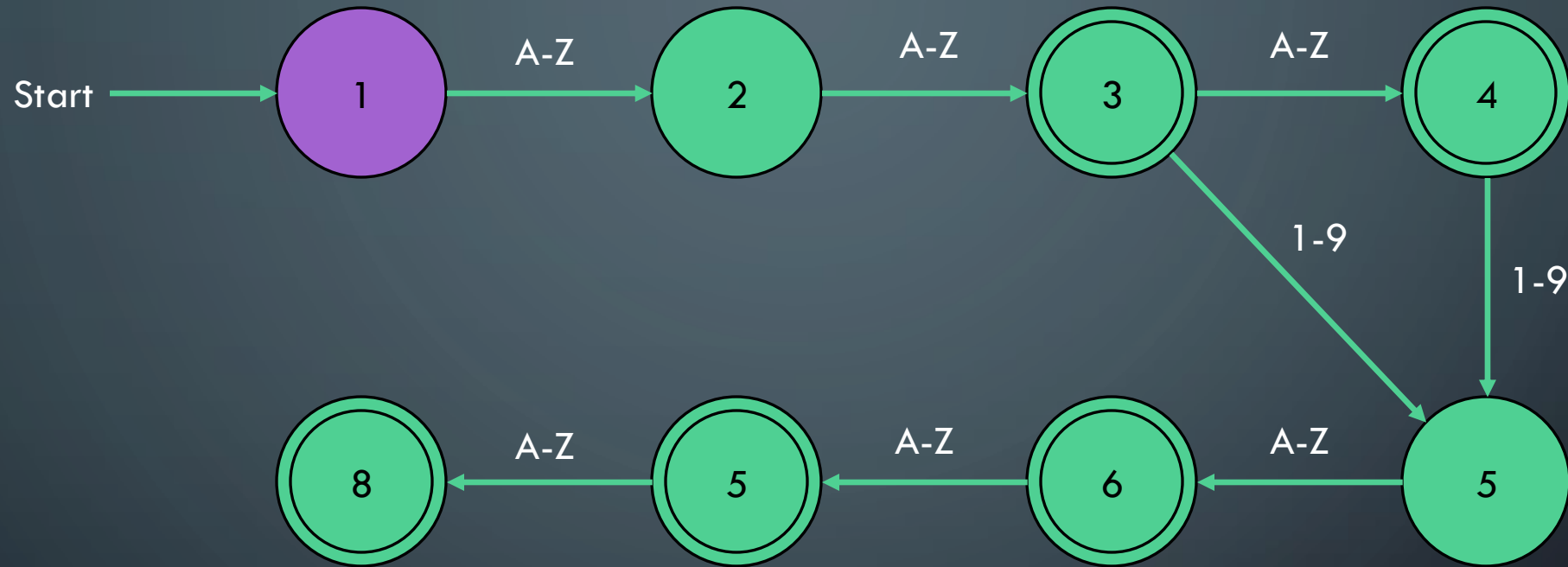
```
mrf8t
as8f
afde9d
```

ALL DFA CAN BE REDUCED TO SAT



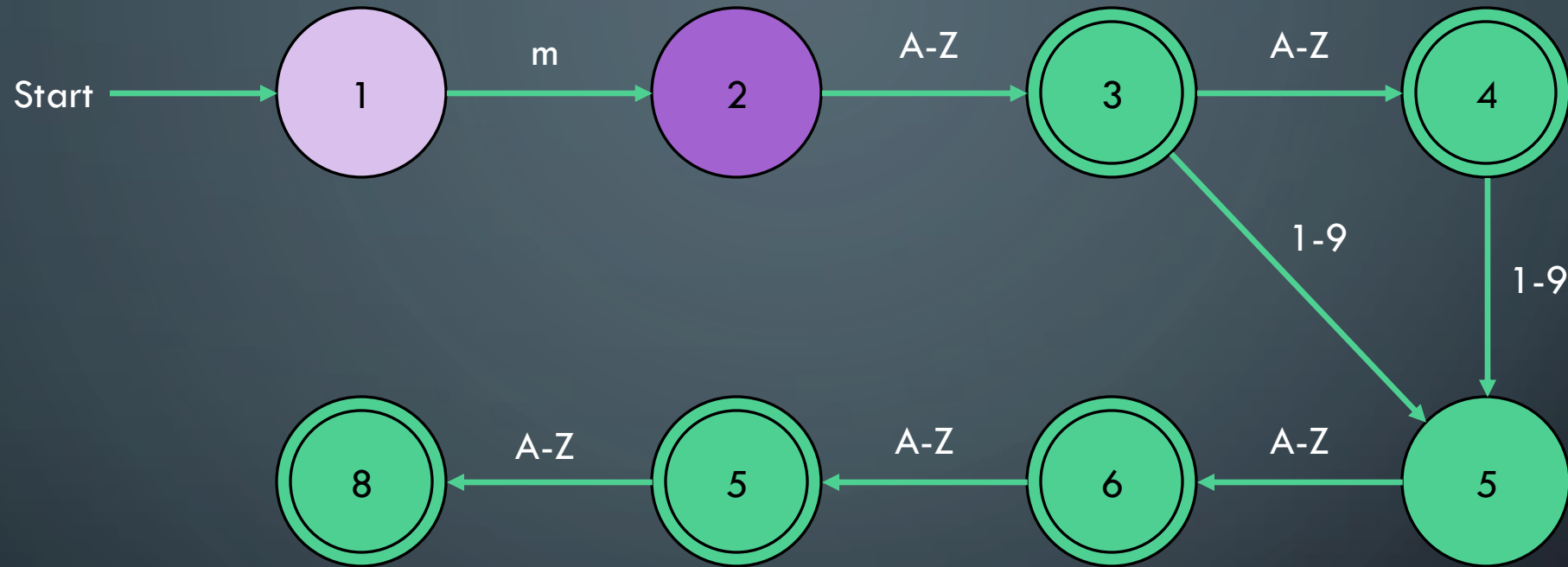
Example Input: mrf8t

ALL DFA CAN BE REDUCED TO SAT



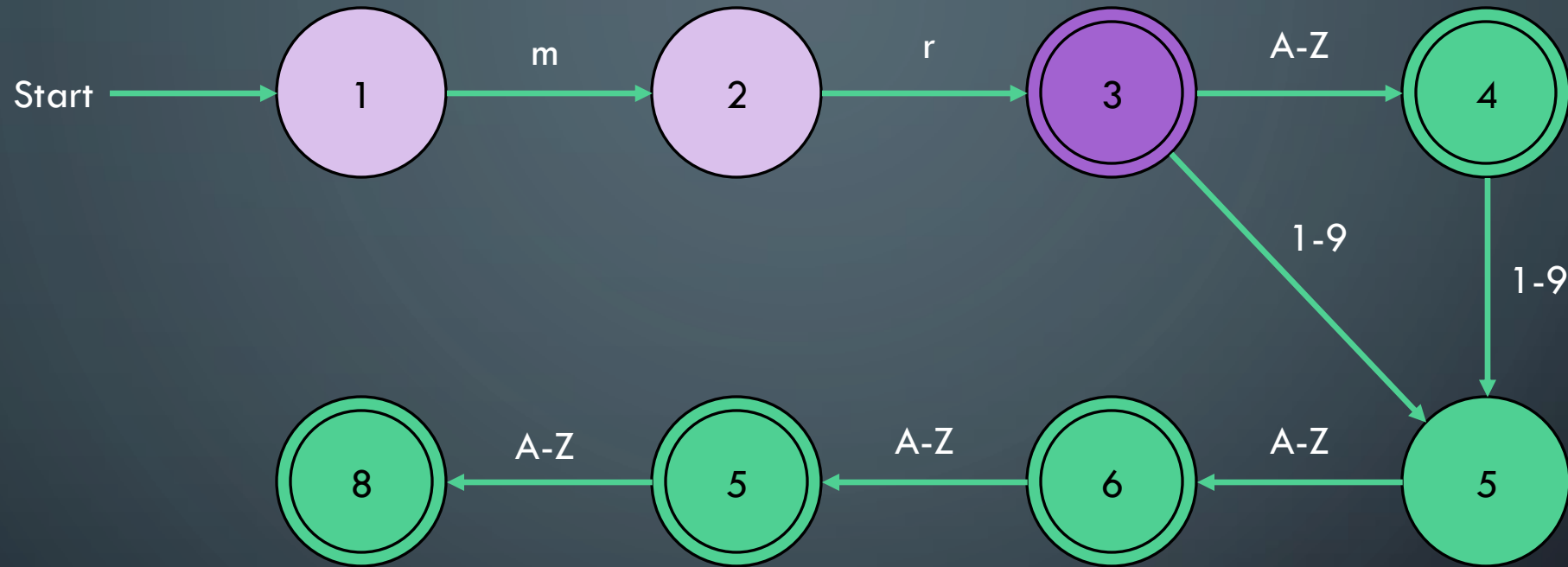
Example Input: mrf8t

ALL DFA CAN BE REDUCED TO SAT



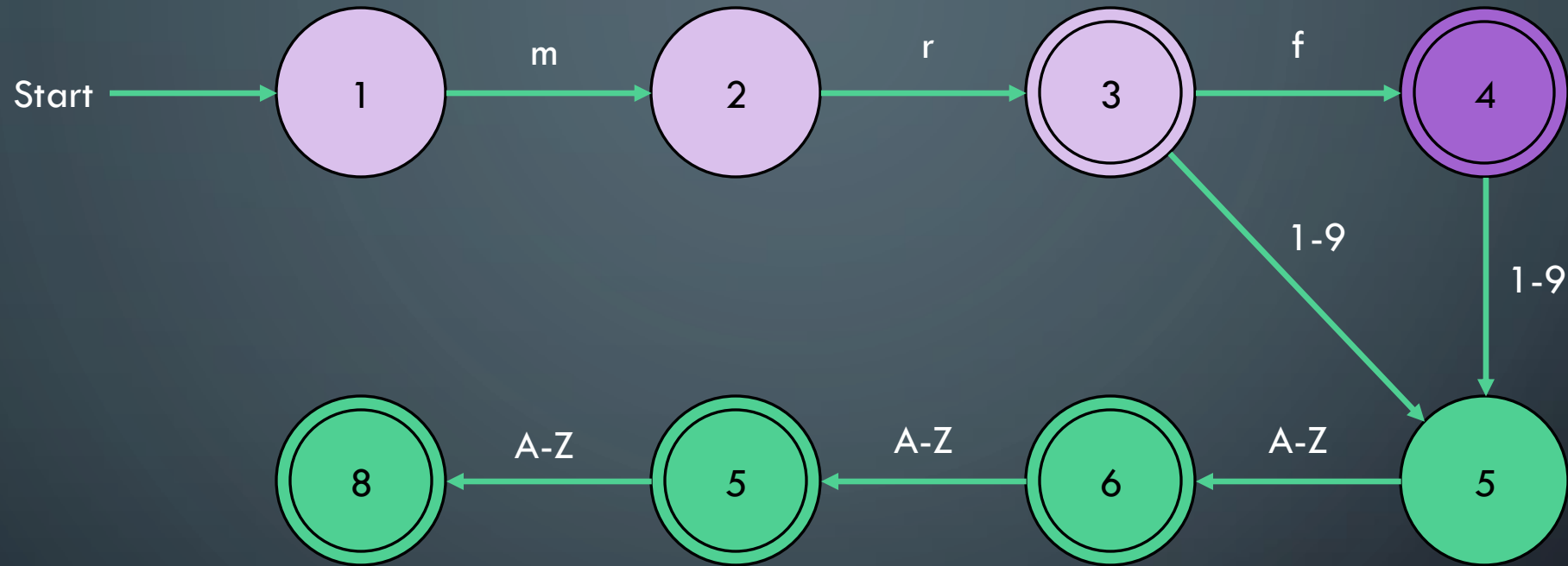
Example Input: mrrf8t

ALL DFA CAN BE REDUCED TO SAT



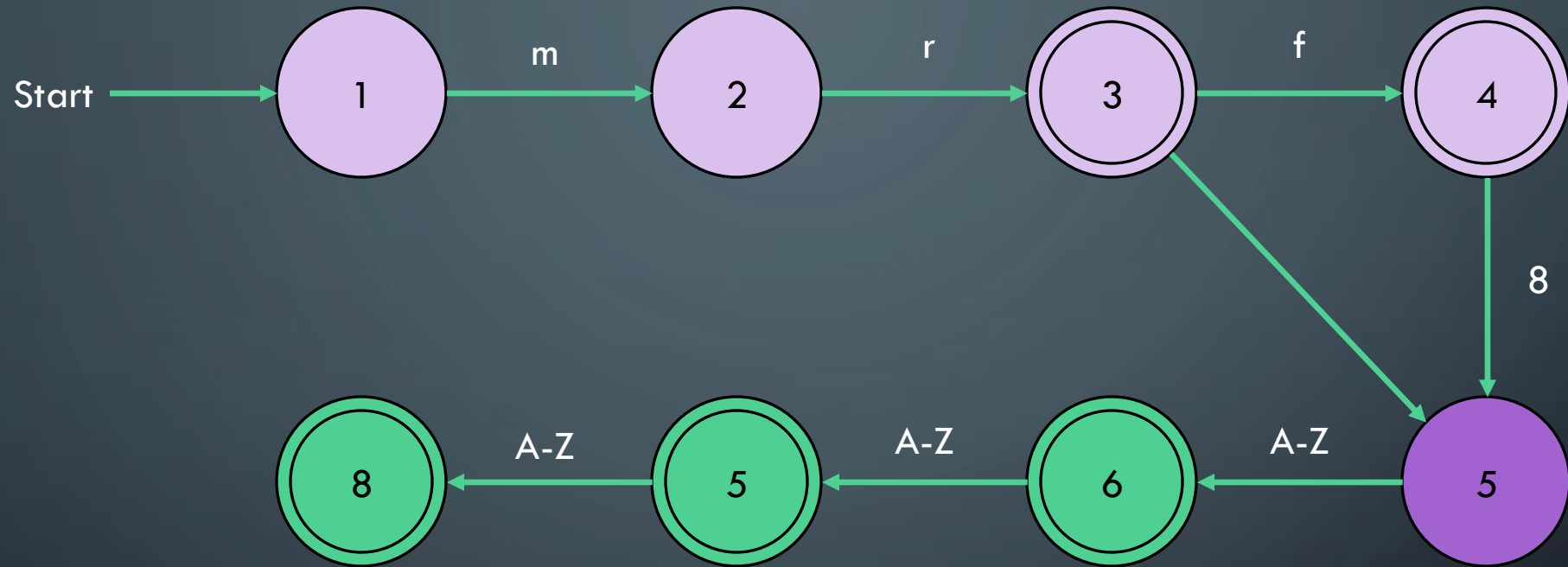
Example Input: `mr`f8t

ALL DFA CAN BE REDUCED TO SAT



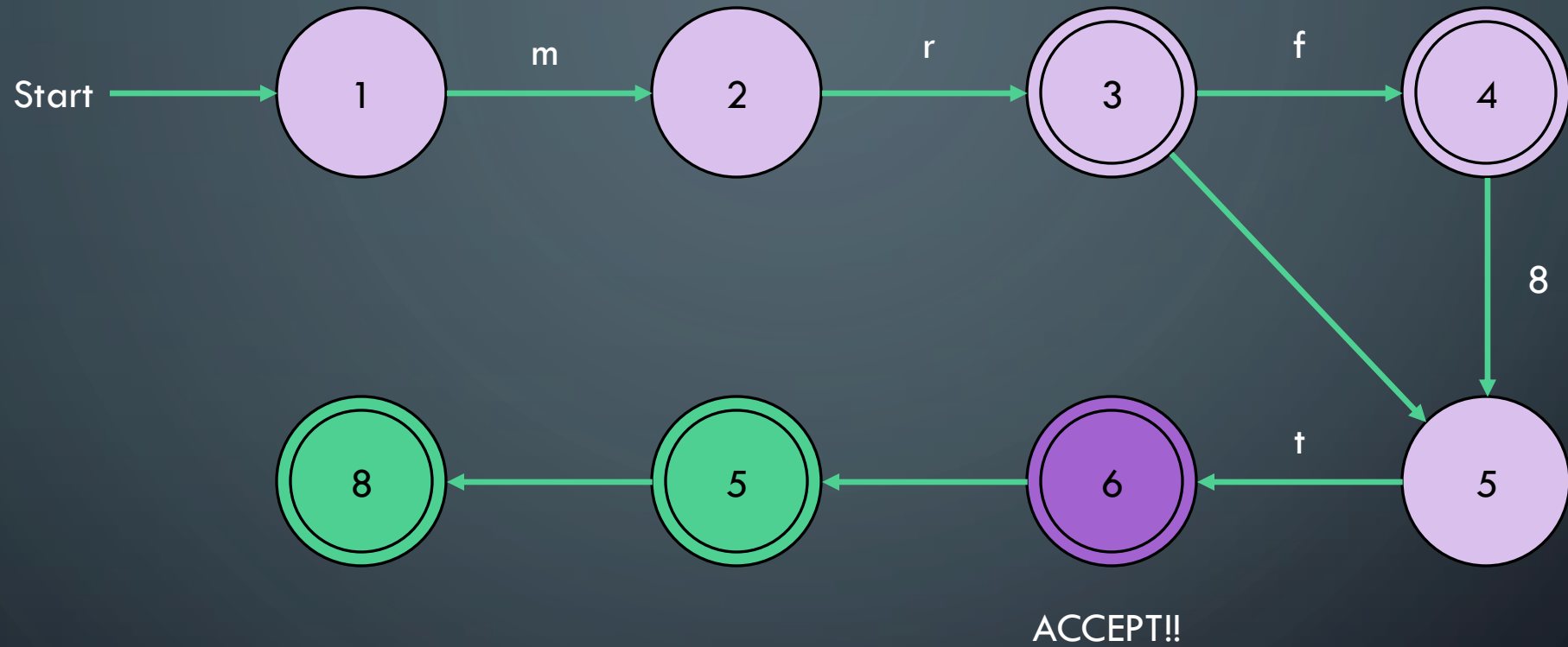
Example Input: `mr`f8t

ALL DFA CAN BE REDUCED TO SAT

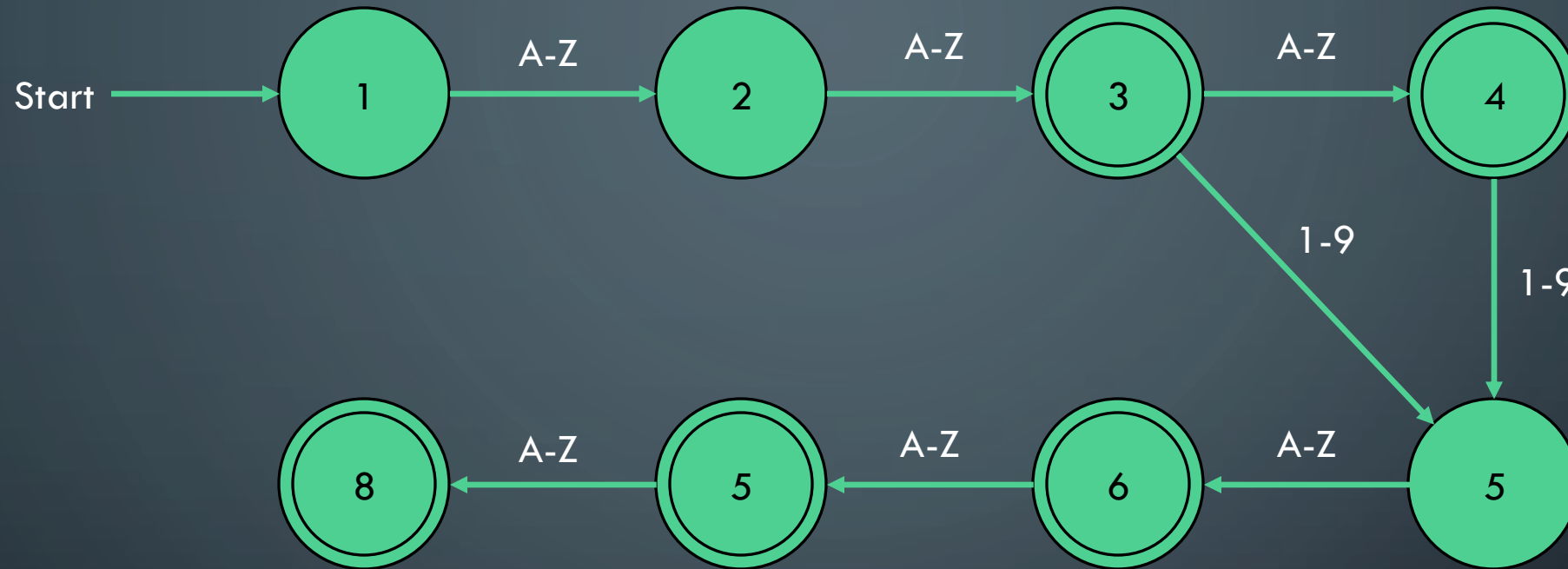


Example Input: `mrf8t`

ALL DFA CAN BE REDUCED TO SAT



A DIFFERENT WAY OF “CODING” THIS...



Definition:

By the way, this is called a **Deterministic Finite Automata**. You'll learn more about these in DMT2! A lot (but not all) programs can be expressed as a **DFA**.

PYTHON CODE

```
def check_uva_userid2(what):
    # returns True if the passed parameter is a valid UVA userid, else False

    state_table = [
        # from each state, where to go on an 'l', 'd', and 'o', respectively
        [1, # we numbered our states from 1, so we burn spot 0
        [2, None, None], # state 1 goes to 2 on a letter
        [3, None, None], # state 2 goes to 3 on a letter
        [4, 5, None], # state 3 goes to 4 on a letter and 5 on a digit
        [None, 5, None], # state 4 goes to 5 on a digit
        [6, None, None], # state 5 goes to 6 on a letter
        [7, None, None], # state 6 goes to 7 on a letter
        [8, None, None], # state 7 goes to 8 on a letter
        [None, None, None], # no transitions out of state 8
        ]
    final_states = [3, 4, 6, 7, 8]

    chars = list(what.lower())
    state = 1
    while len(chars) > 0:

        # which is whether it's a letter (0), digit (1), or other (2)
        which = 0 if chars[0].isalpha() else 1 if chars[0].isdigit() else 2

        # get the state transition, and verify it's not None
        next_state = state_table[state][which]
        if next_state is None: return False

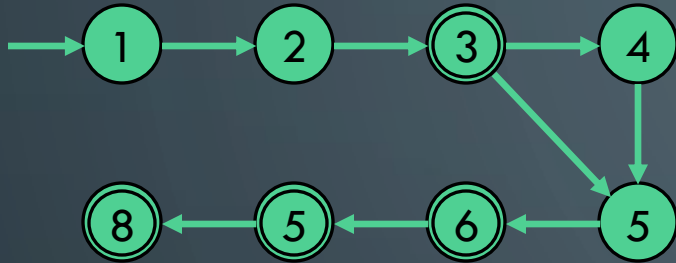
        # transition to that state and pop the input character
        state = next_state
        chars.pop(0)

    # we should have ended in a final state
    return state in final_states

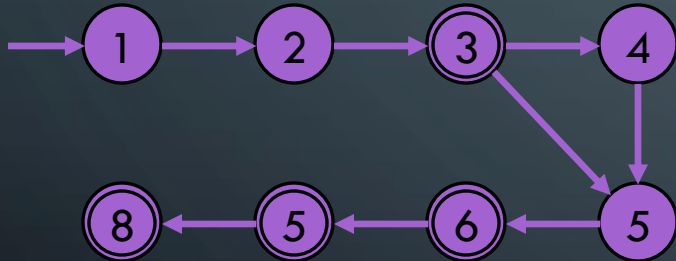
print(check_uva_userid2(input()))
```

Main Point: Coding up these machines is easy, very general, and works for a wide array of problems!!

ASIDE: MORE MACHINE TYPES

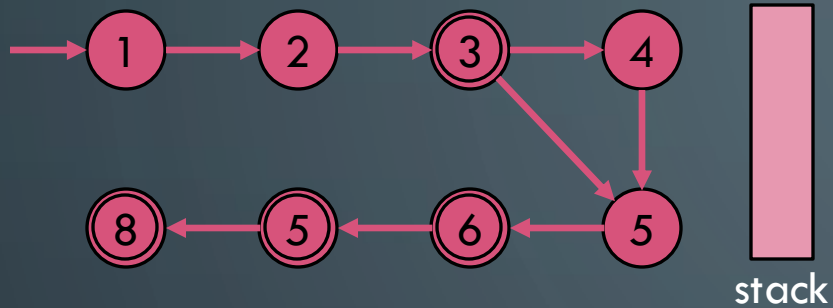


Deterministic Finite Automata. Machine that processes input with no memory (except the current state)

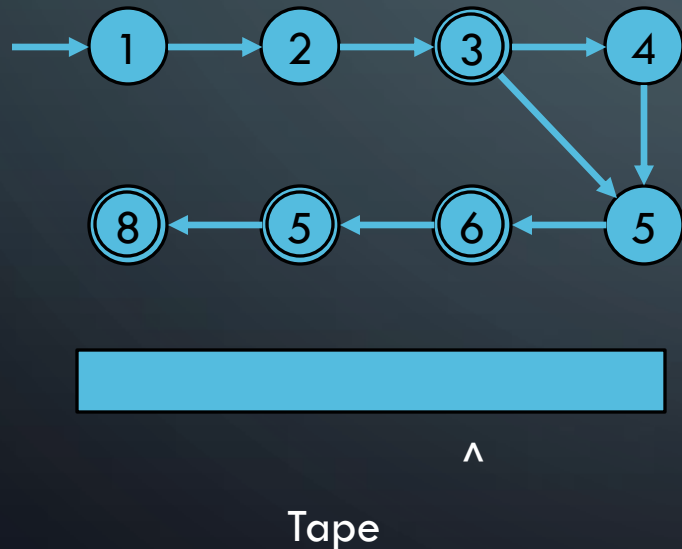


Non-deterministic Finite Automata. DFA that can be in multiple states at once (more in a moment)

ASIDE: MORE MACHINE TYPES



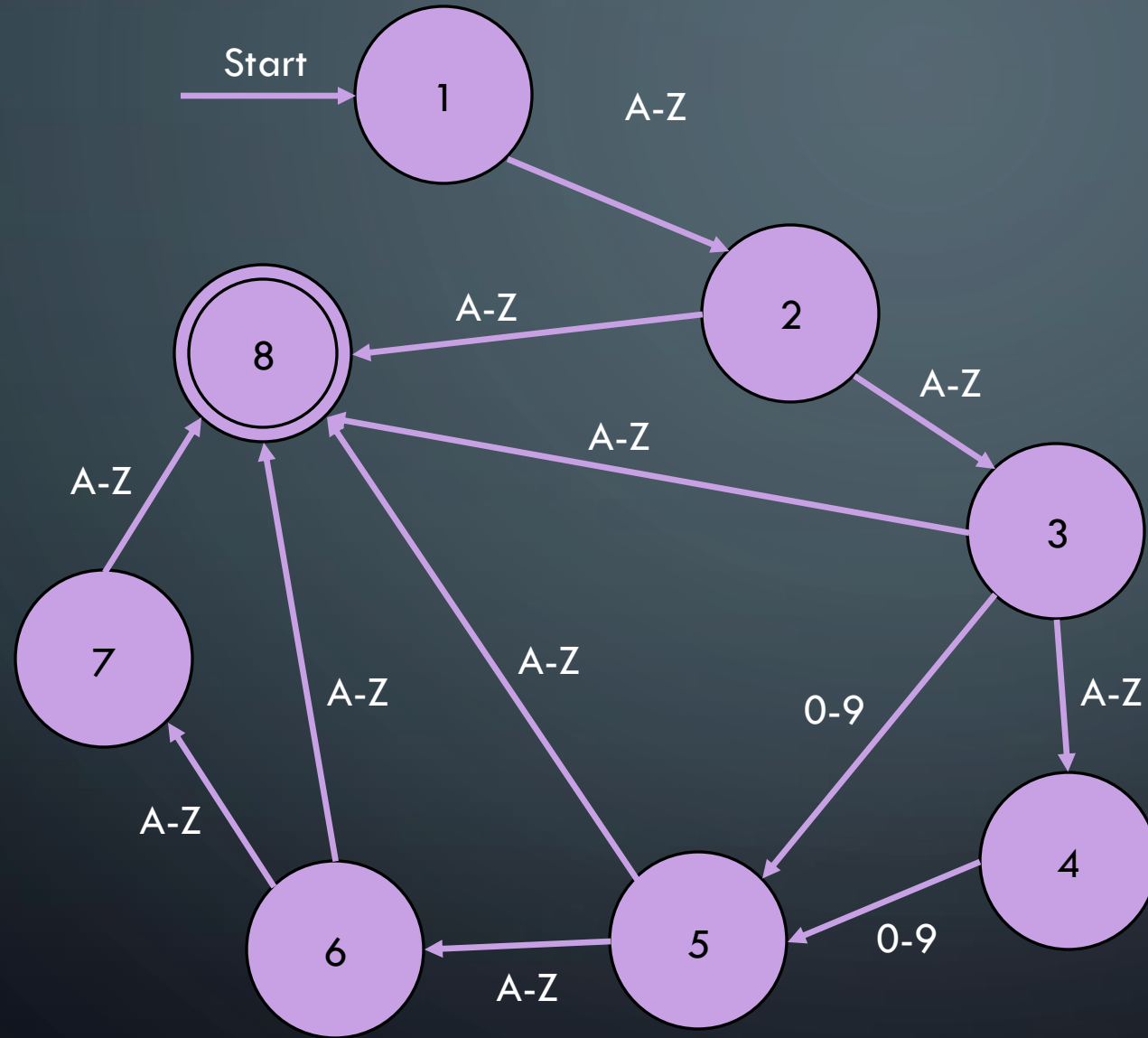
Pushdown Automata. An NFA with a **stack**



Turing Machine. An NFA with a **tape (array)**

You'll see these in a lot more detail in DMT2. They exist, but we are going to stick with the first two for now (to keep things simple)...

UVA ID DETECTION WITH NFA



Try it out:

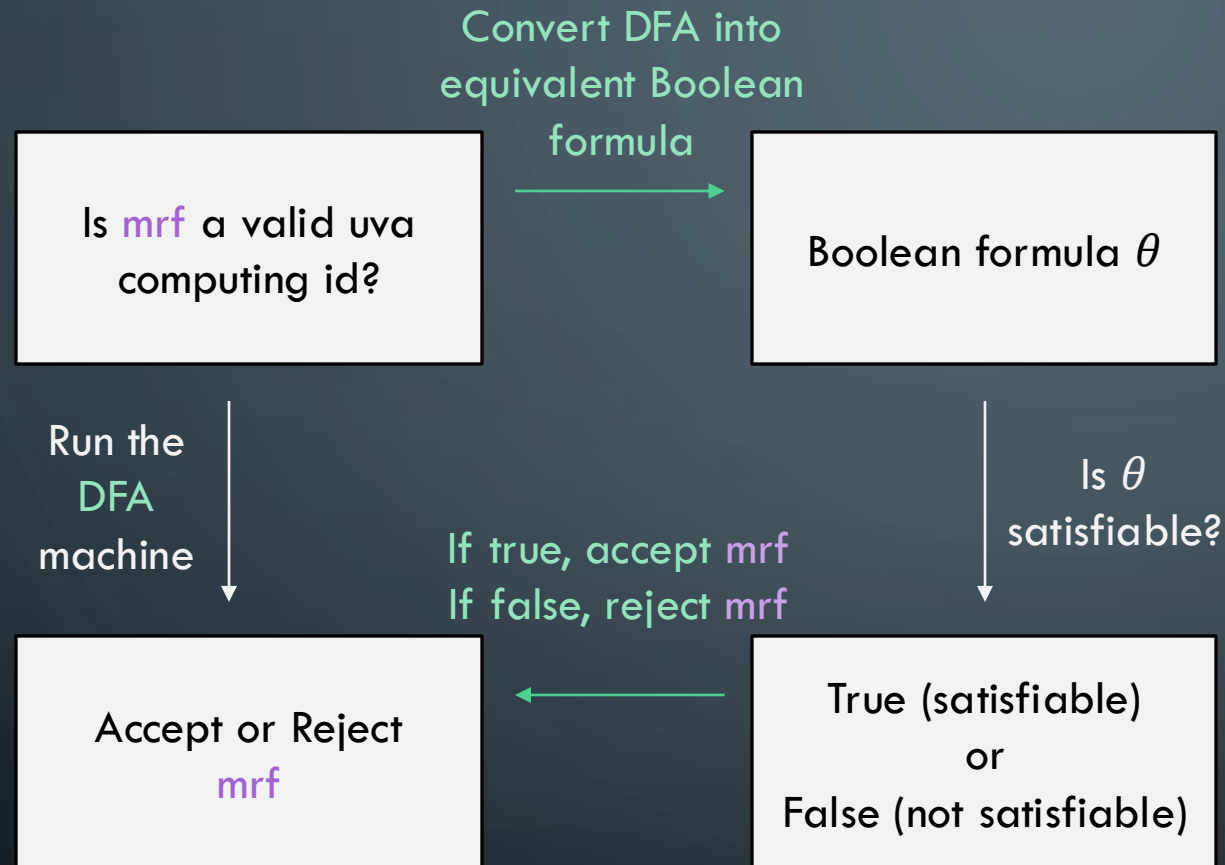
mrf8t
as8f
afde9d

Remember:

String is a valid Uva user id if there is **at least one path** from start to finish on that string.

REDUCTION OF DFA TO SATISFIABILITY

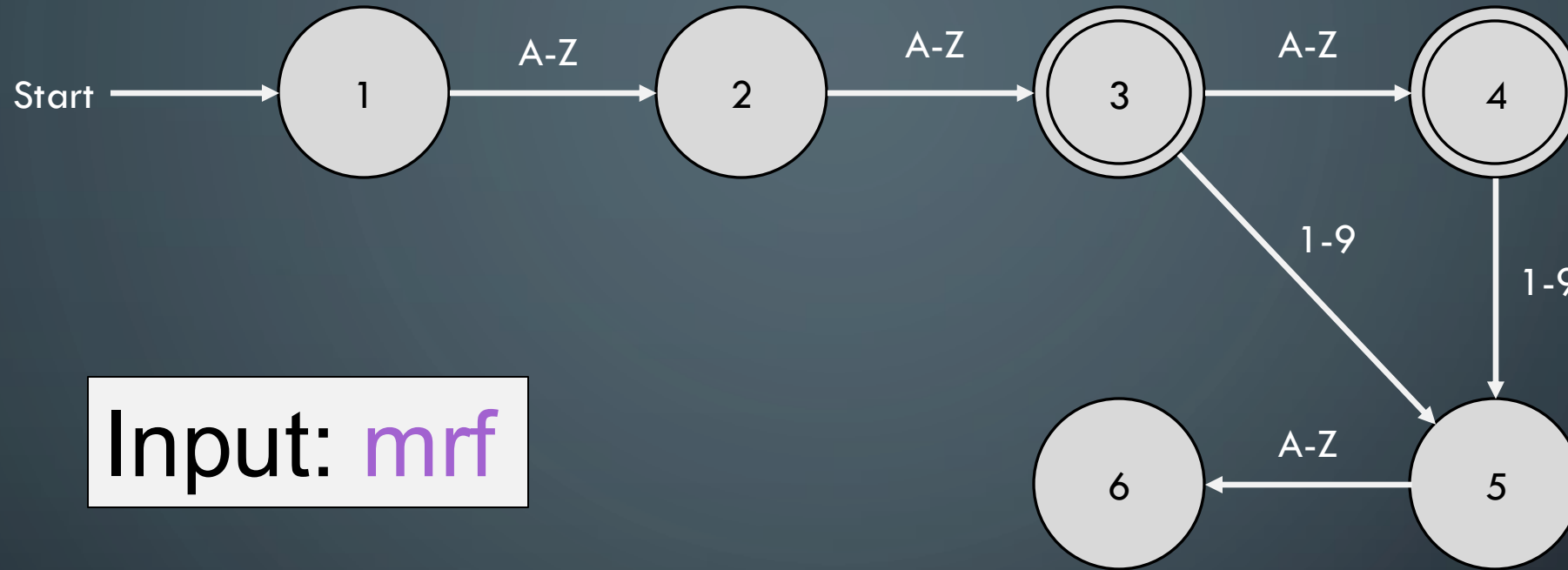
REDUCTION IDEA



Reduction Idea:

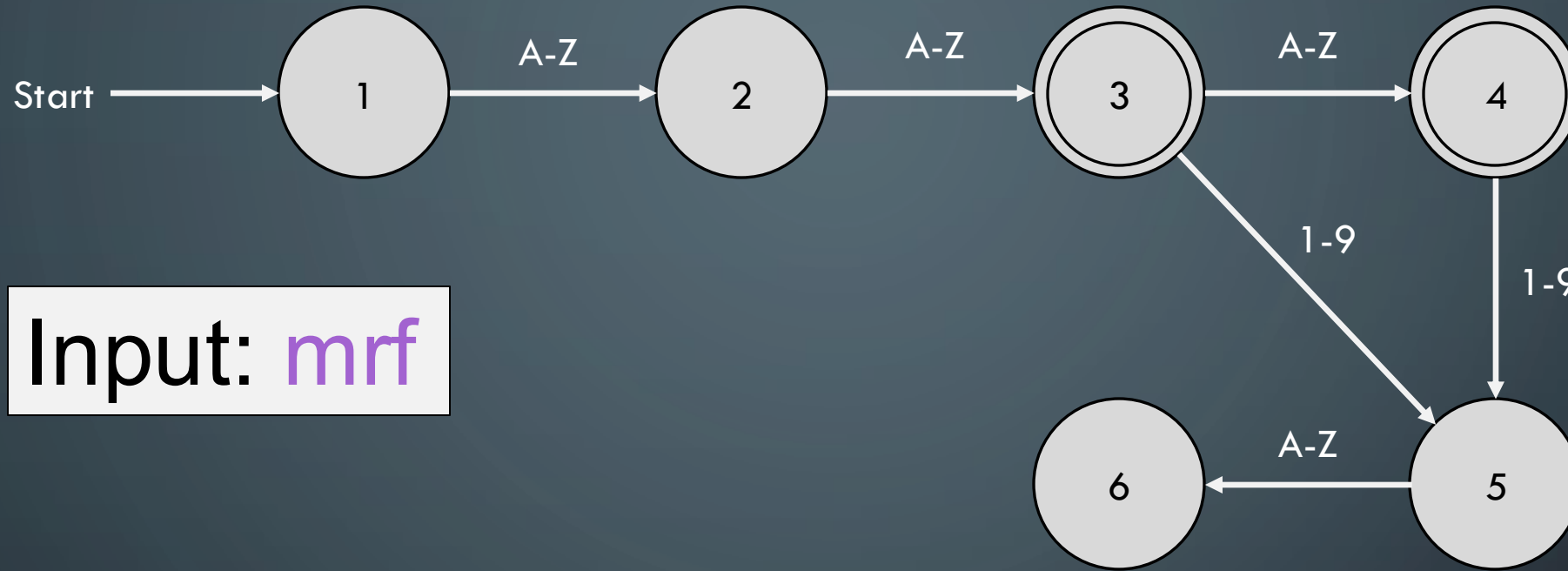
Gives us TWO different ways to “run” this program. We can run the DFA directly OR we can convert into a formula and solve the Boolean formula instead.

CONVERT THIS DFA INTO A FORMULA



Notice we are simplifying the DFA here to make the reduction simpler...now only one letter allowed after the number.

IMPORTANT VARIABLES!



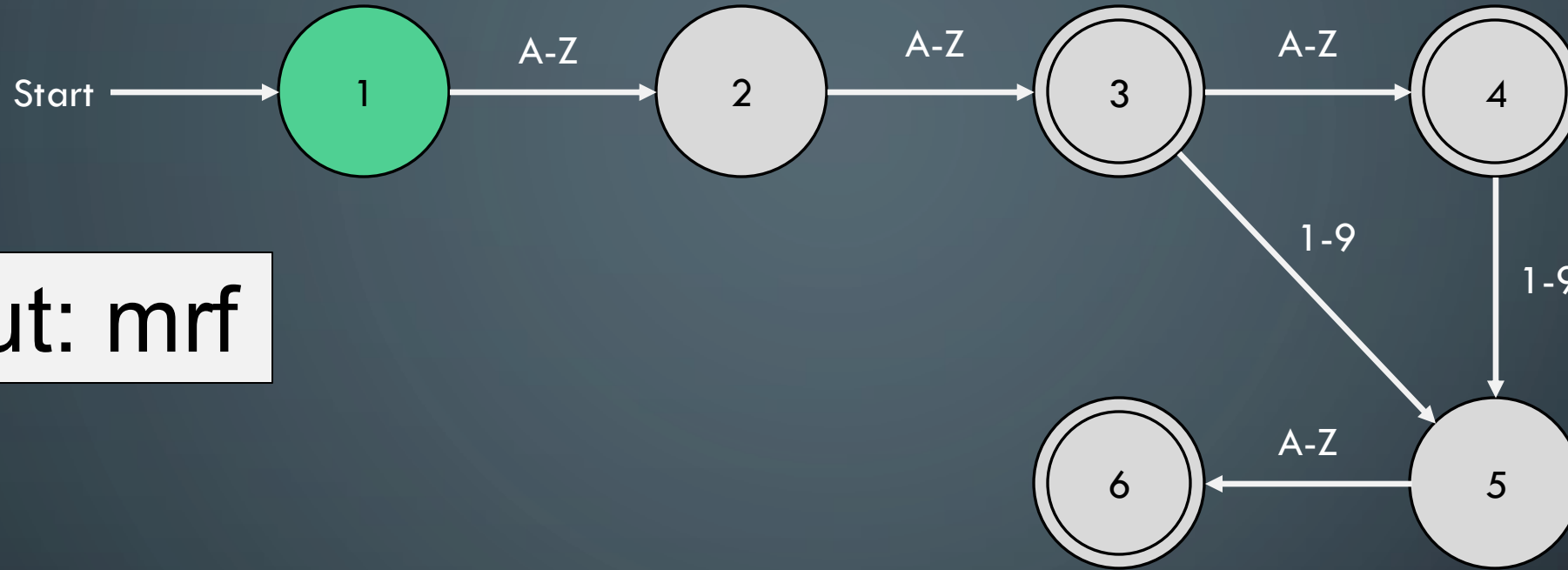
Input: **mrf**

Represents whether machine in state i at step j

$St_{1,1}$	$St_{2,1}$	$St_{3,1}$	$St_{4,1}$	$St_{5,1}$	$St_{6,1}$	$St_{7,1}$	$St_{8,1}$
$St_{1,2}$	$St_{2,2}$	$St_{3,2}$	$St_{4,2}$	$St_{5,2}$	$St_{6,2}$	$St_{7,2}$	$St_{8,2}$
$St_{1,3}$	$St_{2,3}$	$St_{3,3}$	$St_{4,3}$	$St_{5,3}$	$St_{6,3}$	$St_{7,3}$	$St_{8,3}$
$St_{1,4}$	$St_{2,4}$	$St_{3,4}$	$St_{4,4}$	$St_{5,4}$	$St_{6,4}$	$St_{7,4}$	$St_{8,4}$

Notice that each row represents one STEP, so one variable in each row can be true, the others in that row must be false.

IMPORTANT VARIABLES!

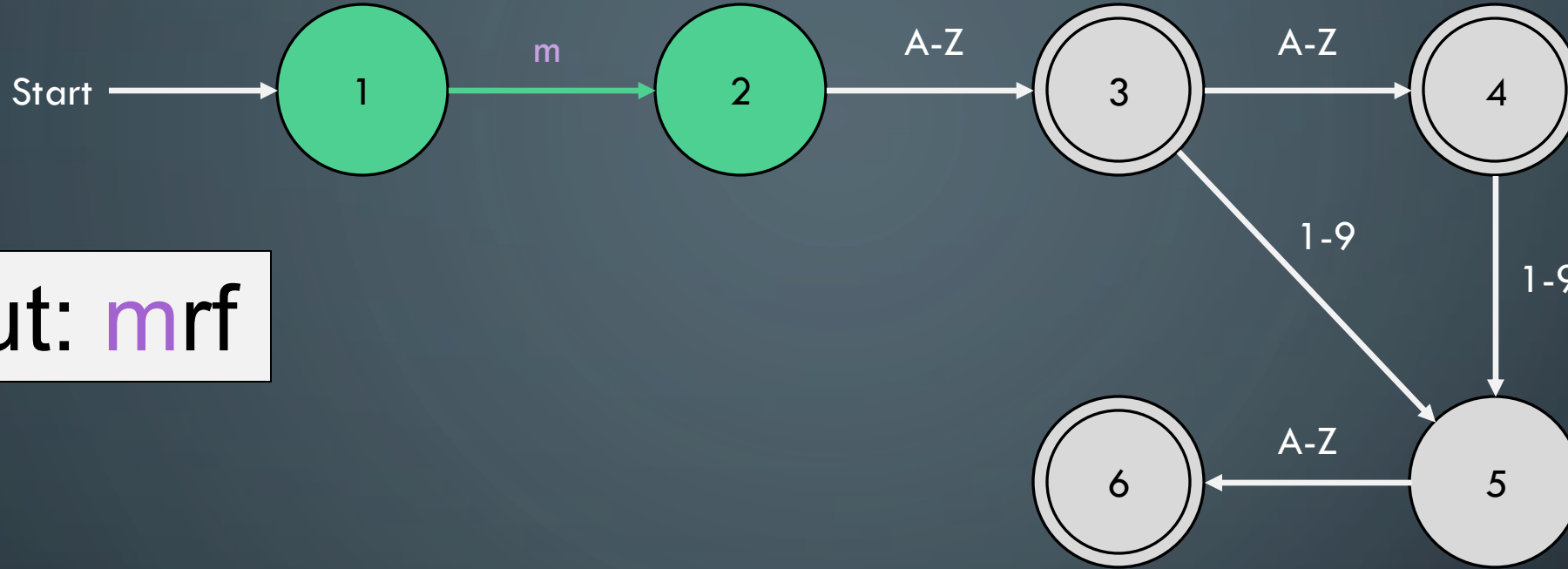


Input: mrf

Represents whether machine in state i at step j

$St_{1,1}$ $St_{2,1}$ $St_{3,1}$ $St_{4,1}$ $St_{5,1}$ $St_{6,1}$ $St_{7,1}$ $St_{8,1}$
 $St_{1,2}$ $St_{2,2}$ $St_{3,2}$ $St_{4,2}$ $St_{5,2}$ $St_{6,2}$ $St_{7,2}$ $St_{8,2}$
 $St_{1,3}$ $St_{2,3}$ $St_{3,3}$ $St_{4,3}$ $St_{5,3}$ $St_{6,3}$ $St_{7,3}$ $St_{8,3}$
 $St_{1,4}$ $St_{2,4}$ $St_{3,4}$ $St_{4,4}$ $St_{5,4}$ $St_{6,4}$ $St_{7,4}$ $St_{8,4}$

IMPORTANT VARIABLES!

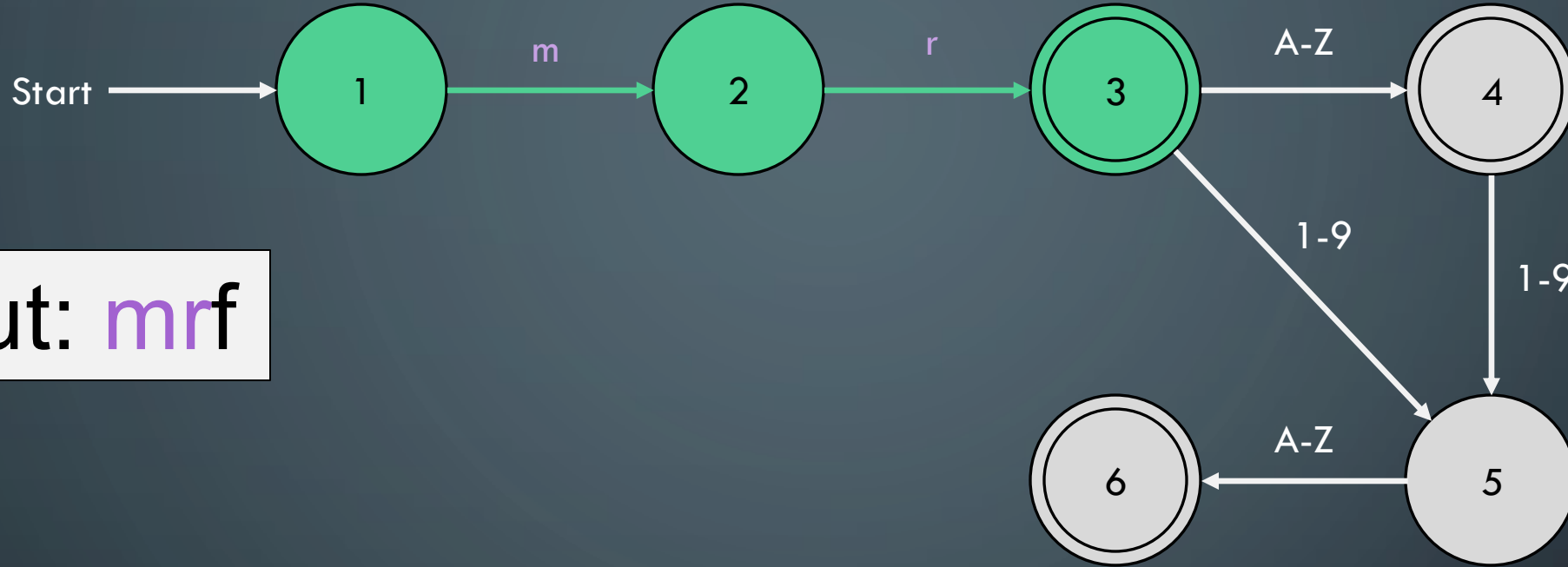


Input: **mrf**

Represents whether machine in state i at step j

$St_{1,1}$ $St_{2,1}$ $St_{3,1}$ $St_{4,1}$ $St_{5,1}$ $St_{6,1}$ $St_{7,1}$ $St_{8,1}$
 $St_{1,2}$ $St_{2,2}$ $St_{3,2}$ $St_{4,2}$ $St_{5,2}$ $St_{6,2}$ $St_{7,2}$ $St_{8,2}$
 $St_{1,3}$ $St_{2,3}$ $St_{3,3}$ $St_{4,3}$ $St_{5,3}$ $St_{6,3}$ $St_{7,3}$ $St_{8,3}$
 $St_{1,4}$ $St_{2,4}$ $St_{3,4}$ $St_{4,4}$ $St_{5,4}$ $St_{6,4}$ $St_{7,4}$ $St_{8,4}$

IMPORTANT VARIABLES!

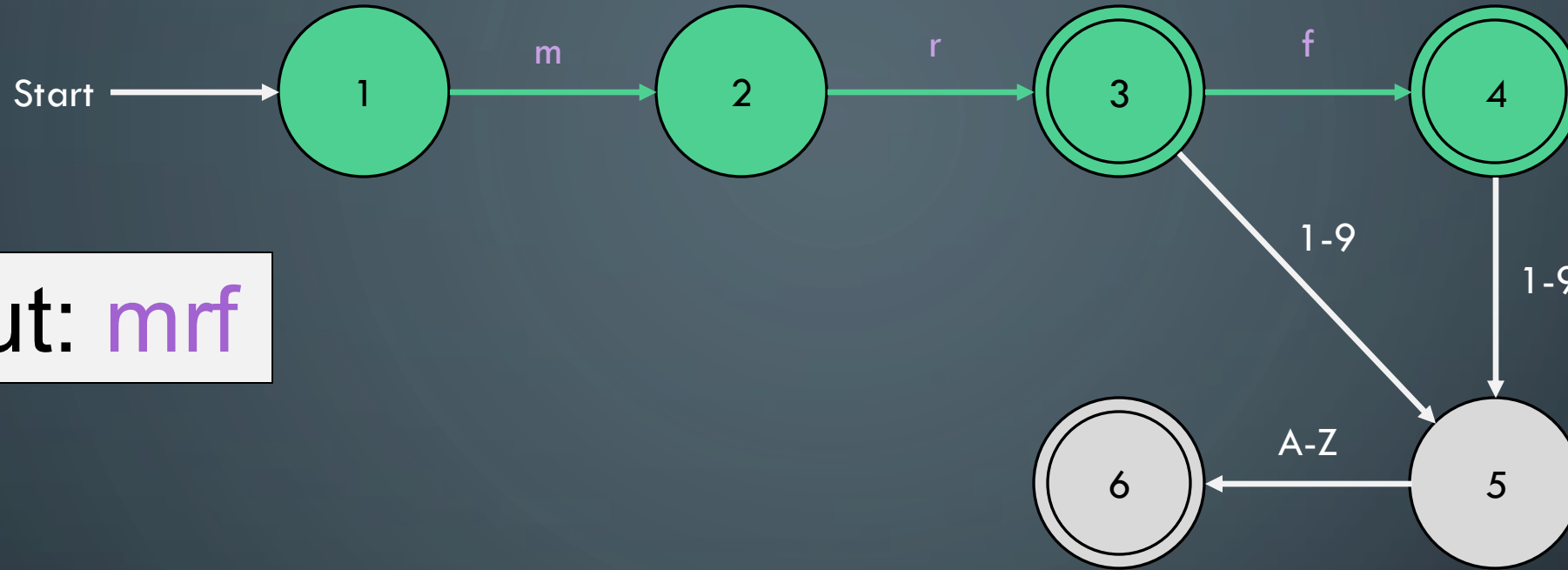


Input: **mrf**

Represents whether machine in state i at step j

$St_{1,1}$	$St_{2,1}$	$St_{3,1}$	$St_{4,1}$	$St_{5,1}$	$St_{6,1}$	$St_{7,1}$	$St_{8,1}$
$St_{1,2}$	$St_{2,2}$	$St_{3,2}$	$St_{4,2}$	$St_{5,2}$	$St_{6,2}$	$St_{7,2}$	$St_{8,2}$
$St_{1,3}$	$St_{2,3}$	$St_{3,3}$	$St_{4,3}$	$St_{5,3}$	$St_{6,3}$	$St_{7,3}$	$St_{8,3}$
$St_{1,4}$	$St_{2,4}$	$St_{3,4}$	$St_{4,4}$	$St_{5,4}$	$St_{6,4}$	$St_{7,4}$	$St_{8,4}$

IMPORTANT VARIABLES!



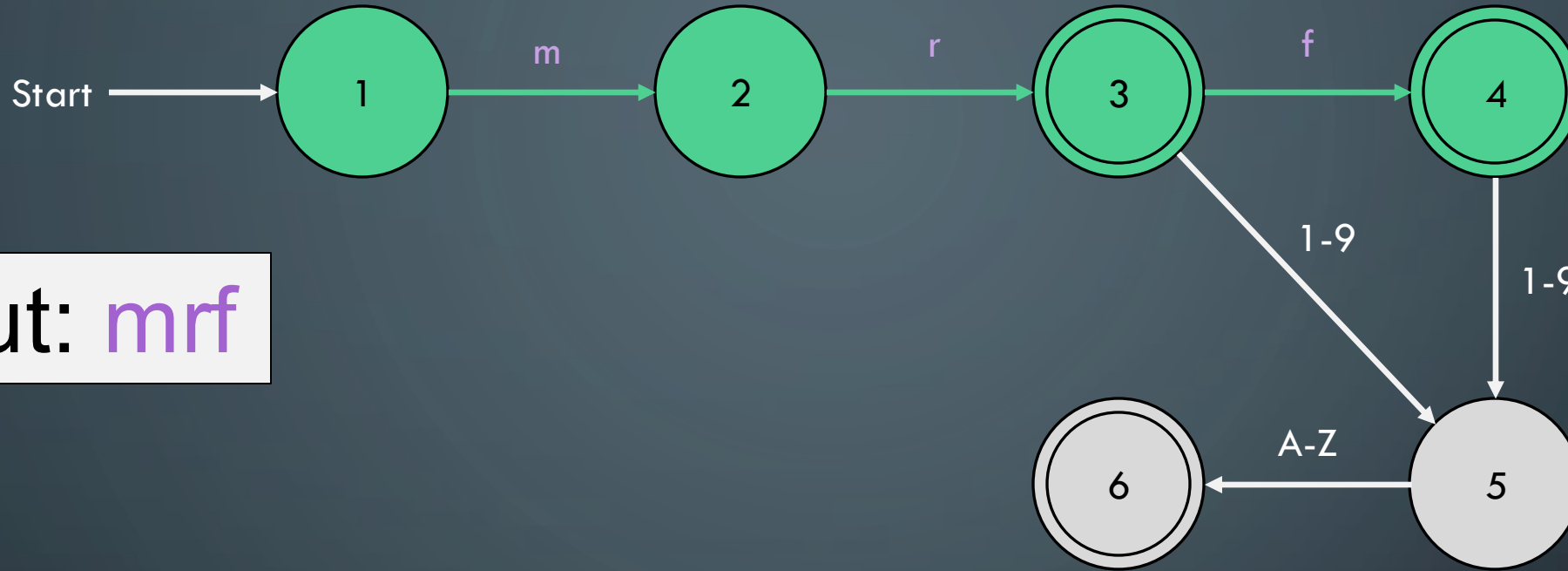
Input: **mrf**

Represents whether machine in state i at step j

$St_{1,1}$ $St_{2,1}$ $St_{3,1}$ $St_{4,1}$ $St_{5,1}$ $St_{6,1}$ $St_{7,1}$ $St_{8,1}$
 $St_{1,2}$ $St_{2,2}$ $St_{3,2}$ $St_{4,2}$ $St_{5,2}$ $St_{6,2}$ $St_{7,2}$ $St_{8,2}$
 $St_{1,3}$ $St_{2,3}$ $St_{3,3}$ $St_{4,3}$ $St_{5,3}$ $St_{6,3}$ $St_{7,3}$ $St_{8,3}$
 $St_{1,4}$ $St_{2,4}$ $St_{3,4}$ $St_{4,4}$ $St_{5,4}$ $St_{6,4}$ $St_{7,4}$ $St_{8,4}$

This is how we eventually want these set when we “run” this machine.

IMPORTANT VARIABLES!



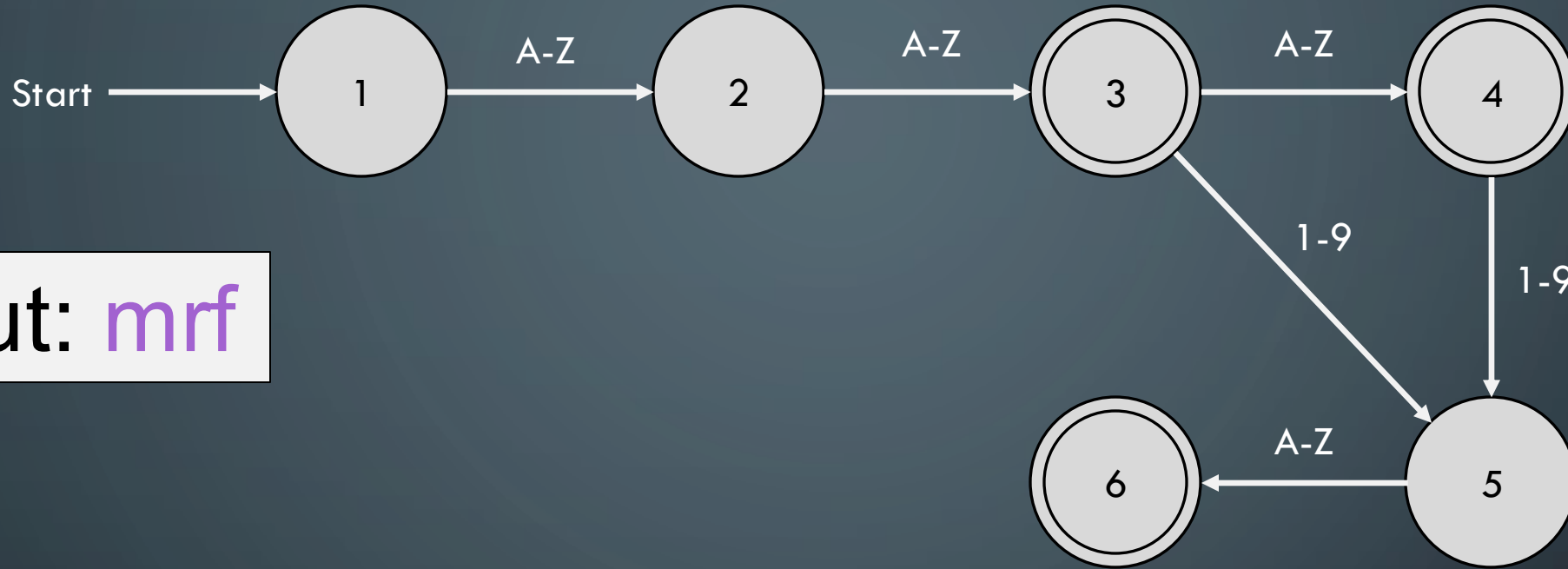
Input: **mrf**

Useful shorthand:

$$\begin{aligned} S_{1,1} &= (\text{green}St_{1,1} \wedge \neg \text{red}St_{2,1} \wedge \neg \text{red}St_{3,1} \wedge \neg \text{red}St_{4,1} \wedge \neg \text{red}St_{5,1} \wedge \neg \text{red}St_{6,1} \wedge \neg \text{red}St_{7,1} \wedge \neg \text{red}St_{8,1}) \\ S_{2,1} &= (\neg \text{red}St_{1,1} \wedge \text{green}St_{2,1} \wedge \neg \text{red}St_{3,1} \wedge \neg \text{red}St_{4,1} \wedge \neg \text{red}St_{5,1} \wedge \neg \text{red}St_{6,1} \wedge \neg \text{red}St_{7,1} \wedge \neg \text{red}St_{8,1}) \\ &\dots \end{aligned}$$

From now on, these SS variables tell us which state we are in at some step AND has terms for not being in all other states at the same time.

IMPORTANT VARIABLES!



Input: **mrf**

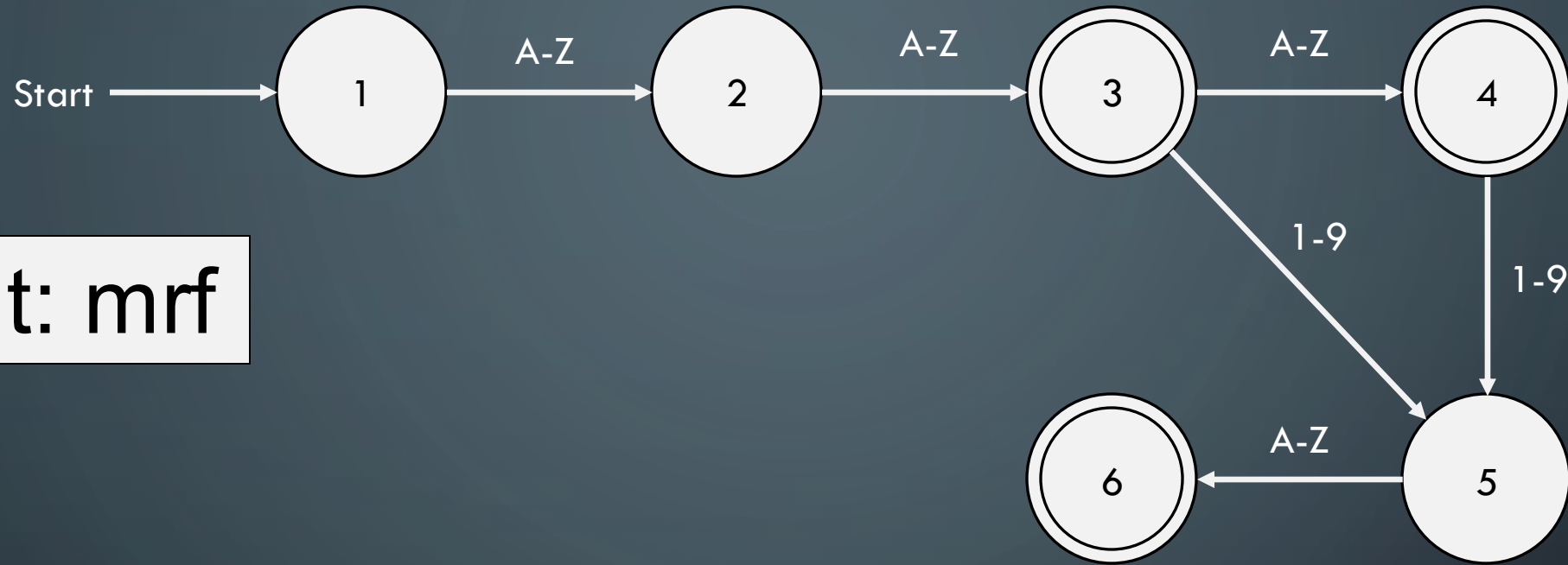
Variables representing the input
being a letter (a) or number (n)

$I_{1,a}$ $I_{1,n}$
 $I_{2,a}$ $I_{2,n}$
 $I_{3,a}$ $I_{3,n}$

So, for our input (**mrf**) we want:

$$(I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n})$$

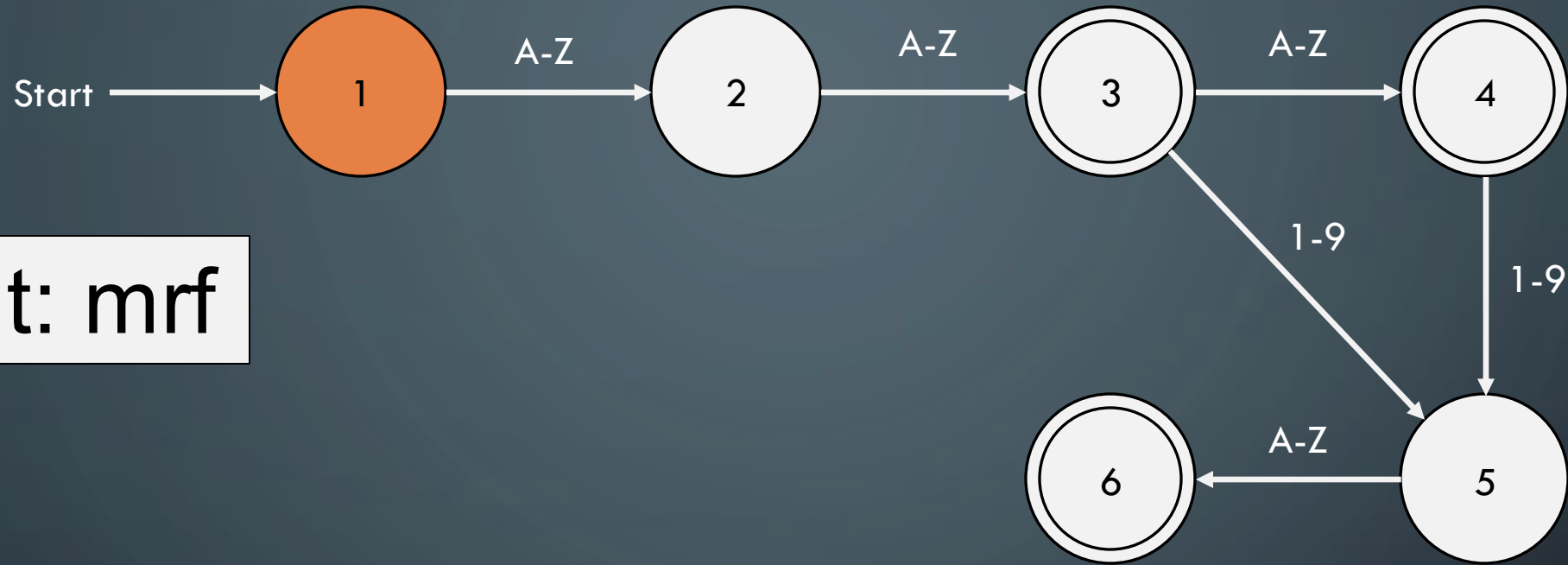
THE CONVERSION



Input: mrf

Ok, we are finally ready to do the conversion of this DFA into a Boolean formula!

THE CONVERSION

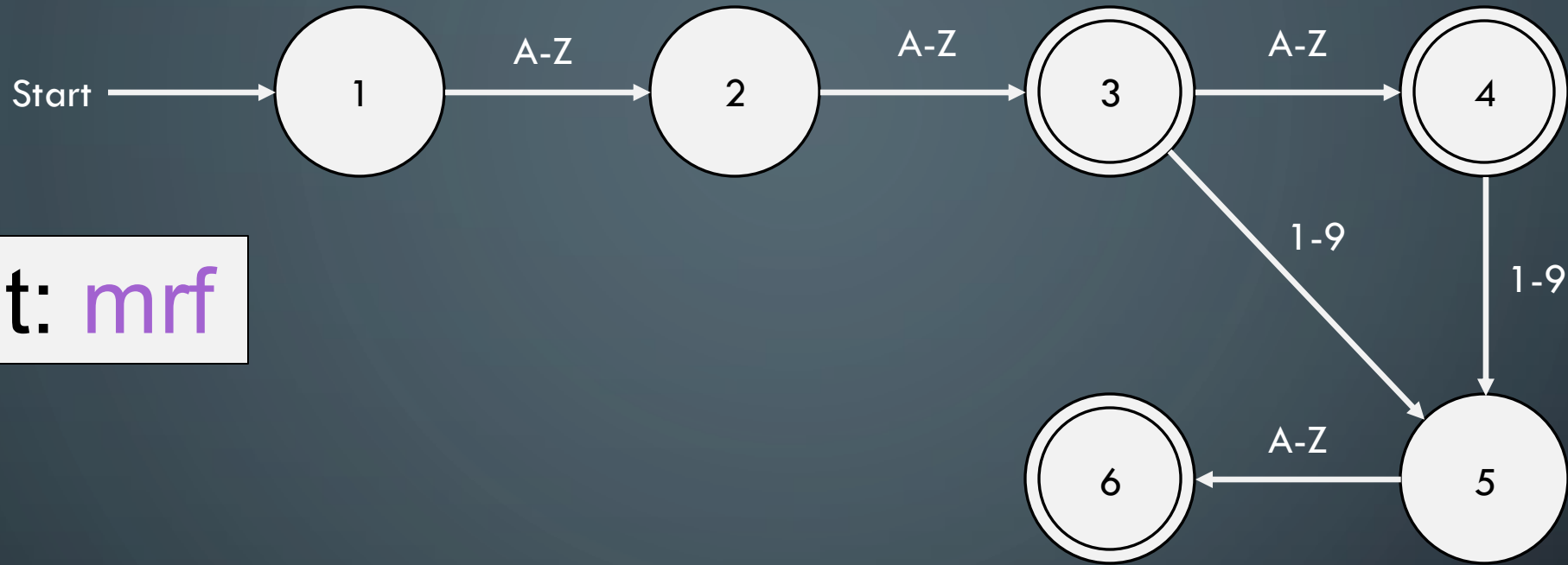


Input: mrf

$$\theta = s_{1,1}$$

Must start in state 1 on first step

THE CONVERSION

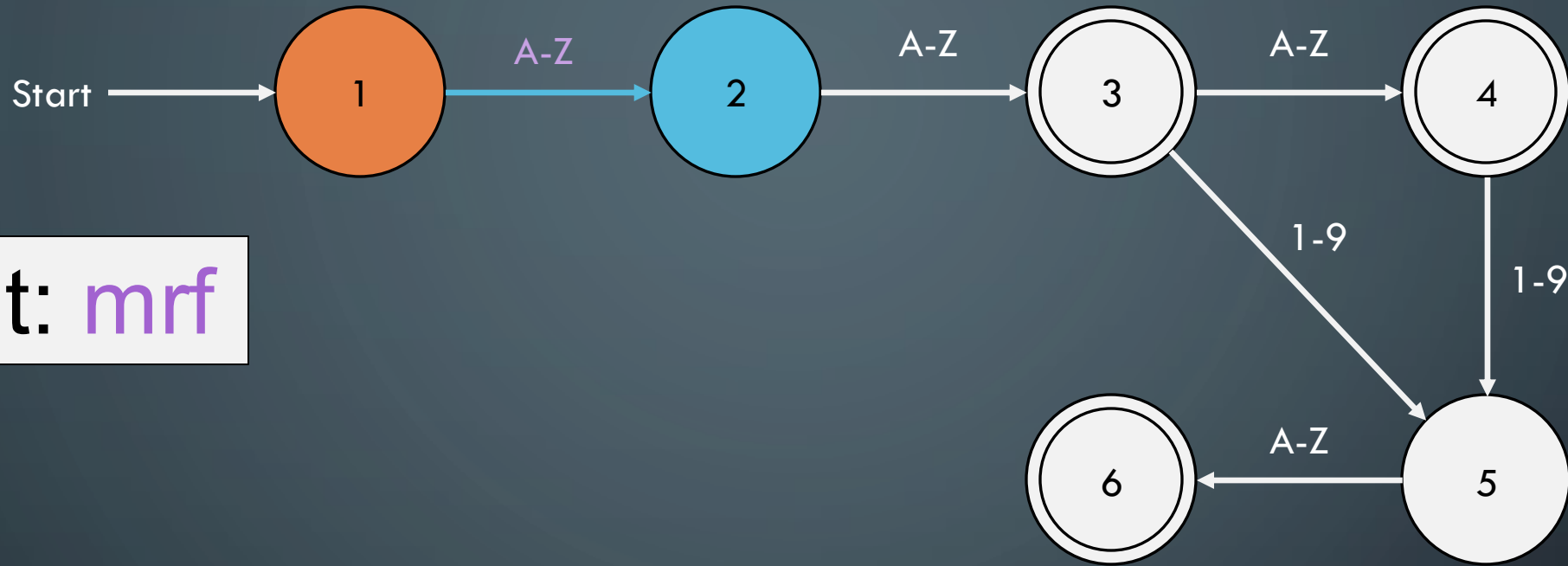


Input: mrf

$$\theta = S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge \dots$$

input string ("mrf") has all three characters being letters (a) and NOT numbers (n)

THE CONVERSION

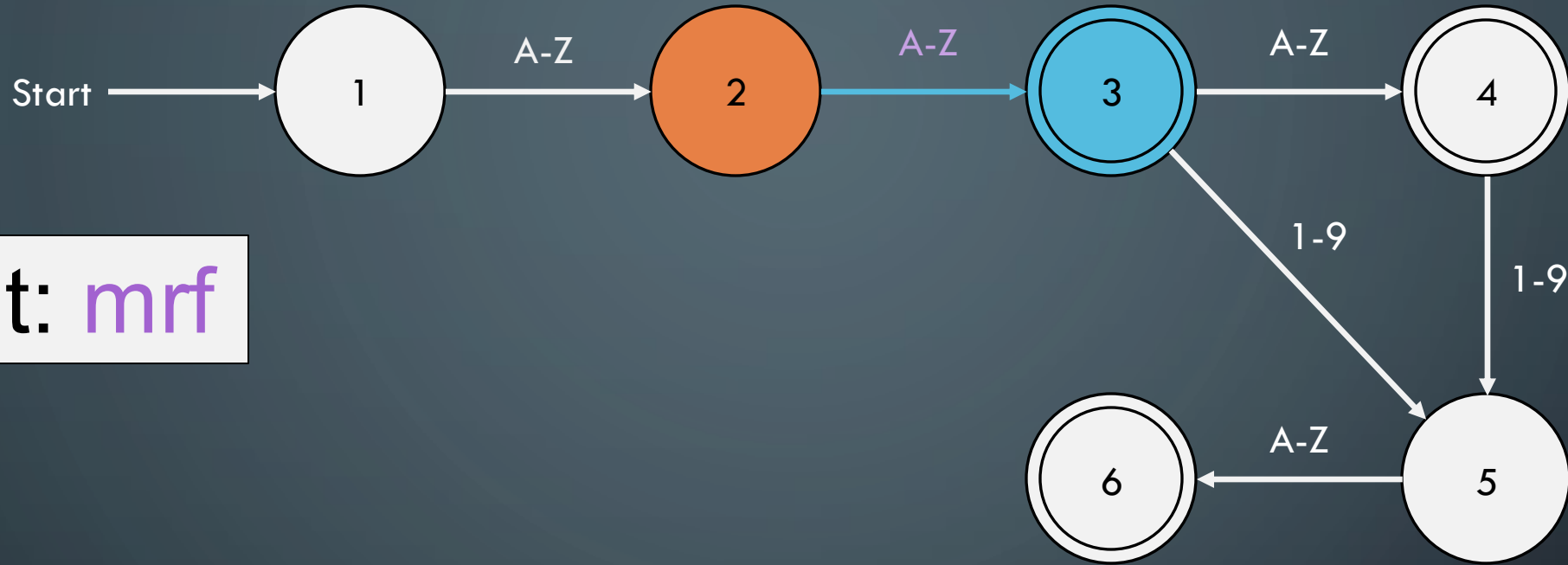


Input: **mrf**

$$\theta = SS_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge \\ (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a}))$$

Since input has exactly three characters, there are three different times we can go from state 1 to state 2.

THE CONVERSION

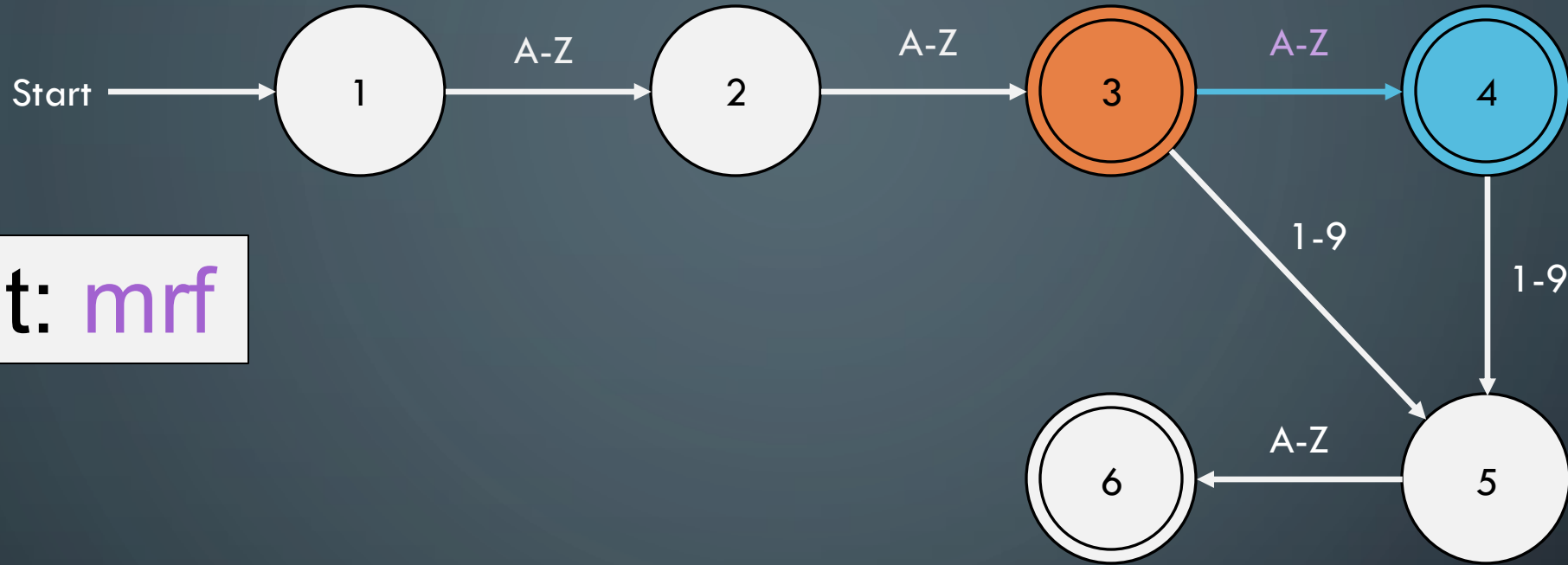


Input: **mrf**

$$\theta = S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge \\ (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge \\ (\textcolor{blue}{S}_{3,2} \leftrightarrow (\textcolor{orange}{S}_{2,1} \wedge \textcolor{purple}{I}_{1,a})) \wedge (\textcolor{blue}{S}_{3,3} \leftrightarrow (\textcolor{orange}{S}_{2,2} \wedge \textcolor{purple}{I}_{2,a})) \wedge (\textcolor{blue}{S}_{3,4} \leftrightarrow (\textcolor{orange}{S}_{2,3} \wedge \textcolor{purple}{I}_{3,a}))$$

Similar, but for the next transition!

THE CONVERSION



Input: mrf

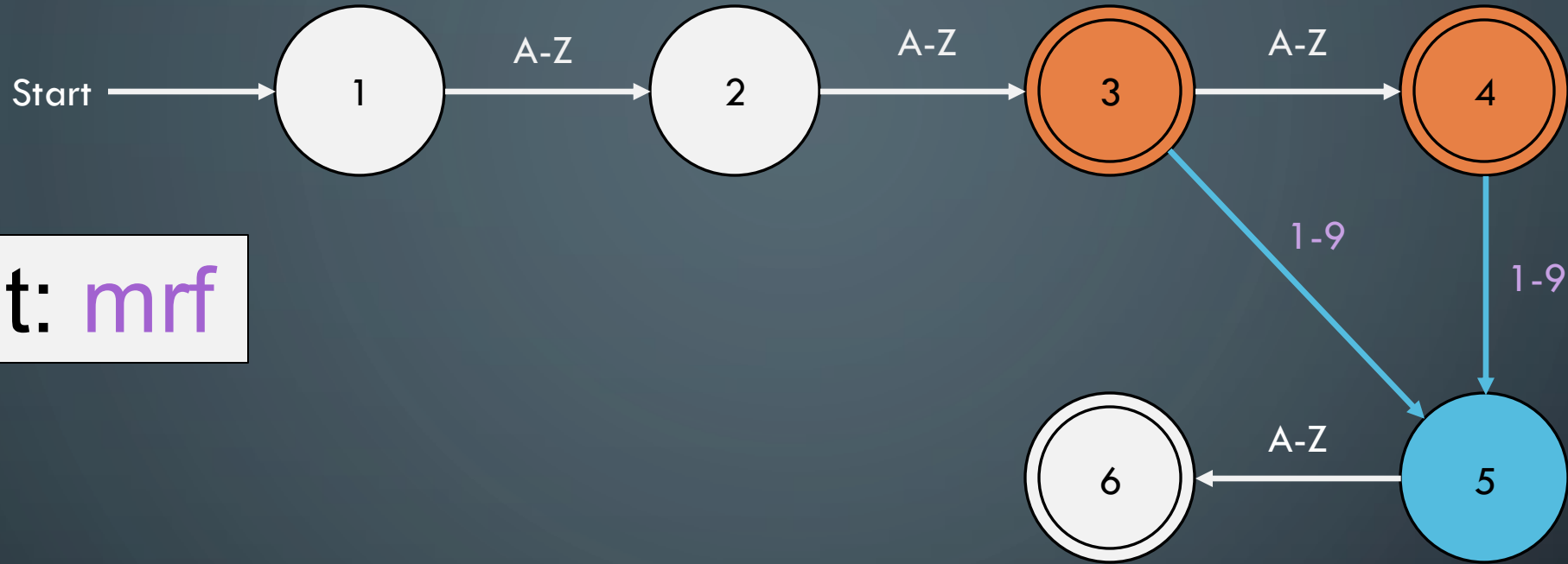
$$\theta = S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge$$

$$(S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge$$

$$(S_{3,2} \leftrightarrow (S_{2,1} \wedge I_{1,a})) \wedge (S_{3,3} \leftrightarrow (S_{2,2} \wedge I_{2,a})) \wedge (S_{3,4} \leftrightarrow (S_{2,3} \wedge I_{3,a})) \wedge$$

$$(S_{4,2} \leftrightarrow (S_{3,1} \wedge I_{1,a})) \wedge (S_{4,3} \leftrightarrow (S_{3,2} \wedge I_{2,a})) \wedge (S_{4,4} \leftrightarrow (S_{3,3} \wedge I_{3,a}))$$

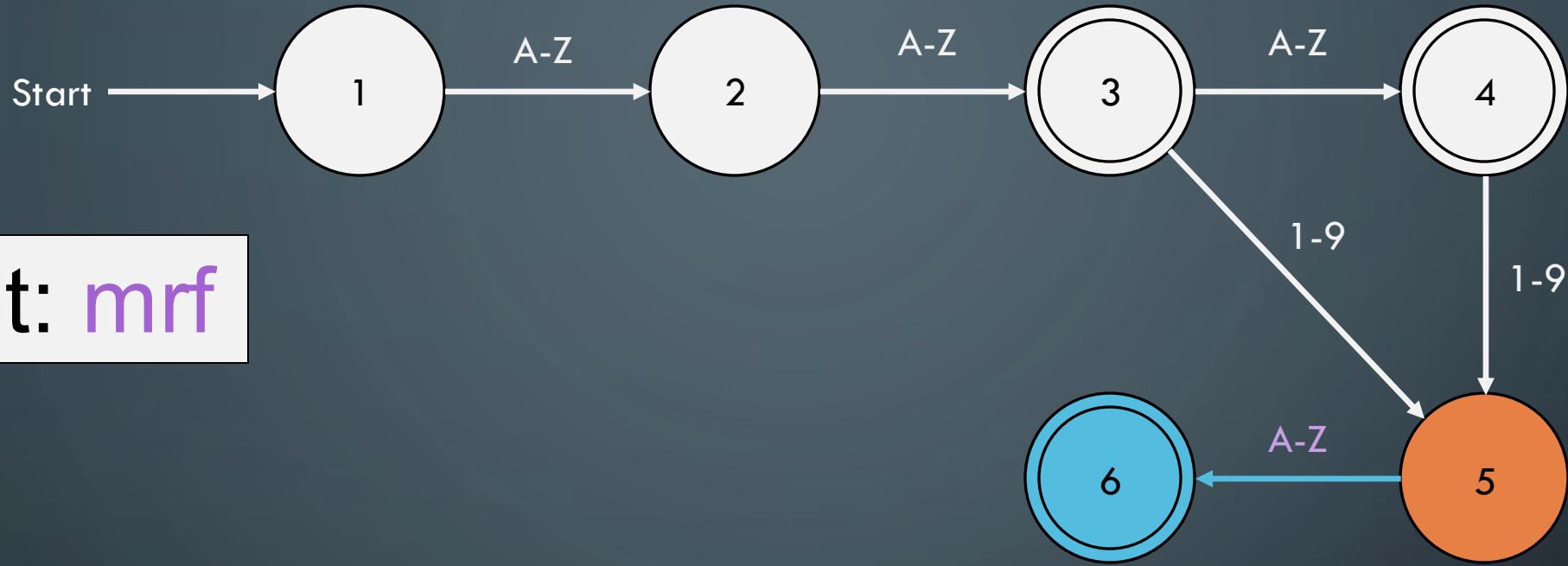
THE CONVERSION



Input: mrf

$$\begin{aligned}
 \theta = & S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge \\
 & (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge \\
 & (S_{3,2} \leftrightarrow (S_{2,1} \wedge I_{1,a})) \wedge (S_{3,3} \leftrightarrow (S_{2,2} \wedge I_{2,a})) \wedge (S_{3,4} \leftrightarrow (S_{2,3} \wedge I_{3,a})) \wedge \\
 & (S_{4,2} \leftrightarrow (S_{3,1} \wedge I_{1,a})) \wedge (S_{4,3} \leftrightarrow (S_{3,2} \wedge I_{2,a})) \wedge (S_{4,4} \leftrightarrow (S_{3,3} \wedge I_{3,a})) \wedge \\
 & (S_{5,2} \leftrightarrow ((S_{3,1} \vee S_{4,1}) \wedge I_{1,n})) \wedge (S_{5,3} \leftrightarrow ((S_{3,2} \vee S_{4,2}) \wedge I_{2,n})) \wedge (S_{5,4} \leftrightarrow ((S_{3,3} \vee S_{4,3}) \wedge I_{3,n}))
 \end{aligned}$$

THE CONVERSION

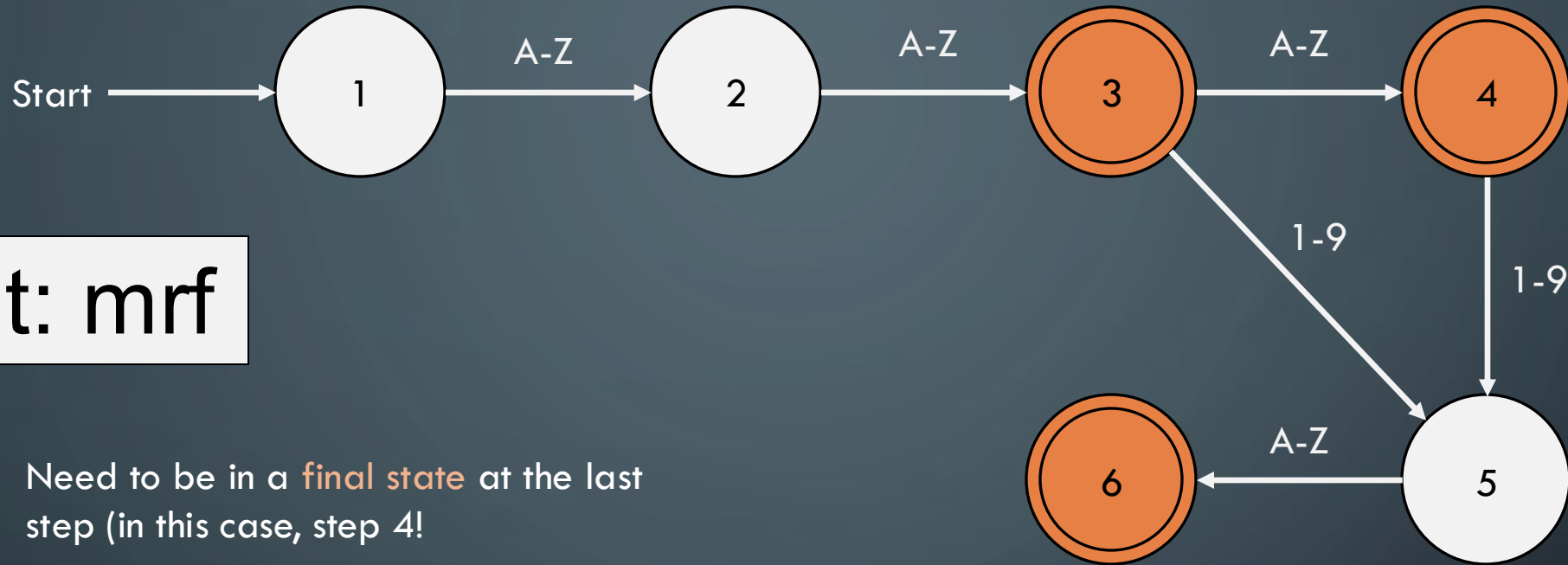


Input: mrf

$$\begin{aligned}
 \theta = & S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge \\
 & (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge \\
 & (S_{3,2} \leftrightarrow (S_{2,1} \wedge I_{1,a})) \wedge (S_{3,3} \leftrightarrow (S_{2,2} \wedge I_{2,a})) \wedge (S_{3,4} \leftrightarrow (S_{2,3} \wedge I_{3,a})) \wedge \\
 & (S_{4,2} \leftrightarrow (S_{3,1} \wedge I_{1,a})) \wedge (S_{4,3} \leftrightarrow (S_{3,2} \wedge I_{2,a})) \wedge (S_{4,4} \leftrightarrow (S_{3,3} \wedge I_{3,a})) \wedge \\
 & (S_{5,2} \leftrightarrow ((S_{3,1} \vee S_{4,1}) \wedge I_{1,n})) \wedge (S_{5,3} \leftrightarrow ((S_{3,2} \vee S_{4,2}) \wedge I_{2,n})) \wedge (S_{5,4} \leftrightarrow ((S_{3,3} \vee S_{4,3}) \wedge I_{3,n})) \wedge \\
 & (S_{6,2} \leftrightarrow (S_{5,1} \wedge I_{1,a})) \wedge (S_{6,3} \leftrightarrow (S_{5,2} \wedge I_{2,a})) \wedge (S_{6,4} \leftrightarrow (S_{5,3} \wedge I_{3,a}))
 \end{aligned}$$

THE CONVERSION

Input: mrf

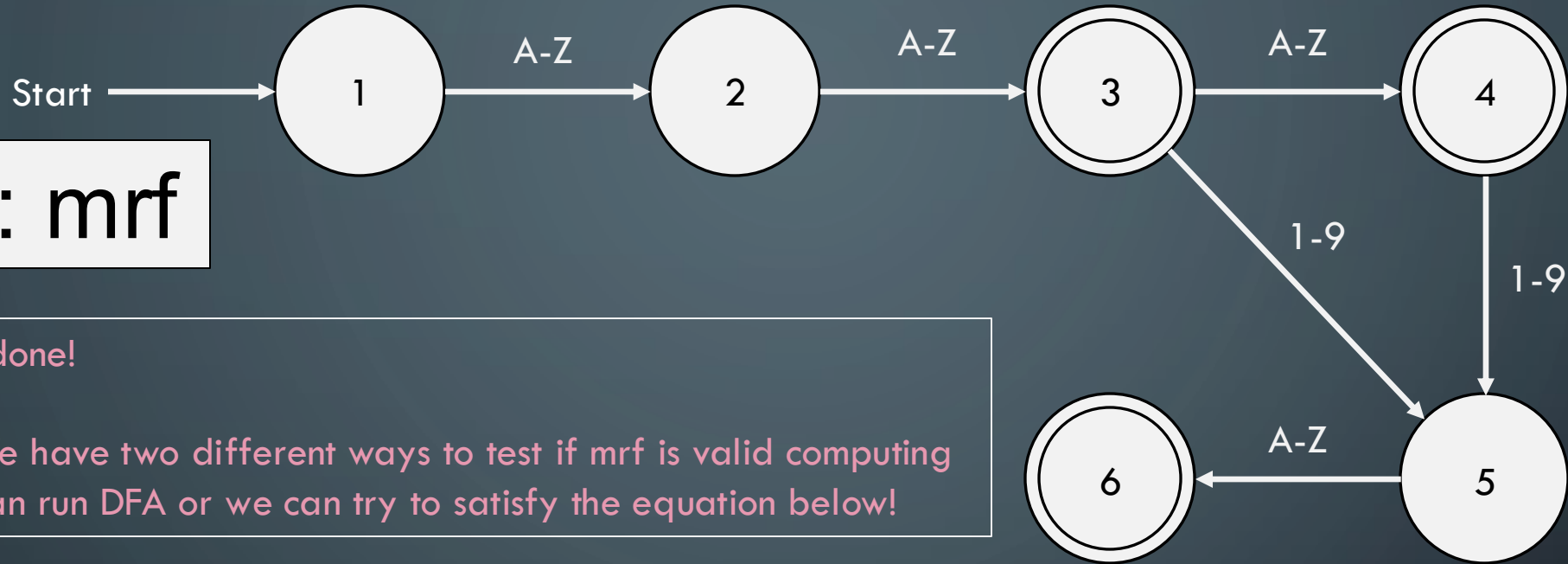


Need to be in a **final state** at the last step (in this case, step 4!)

$$\begin{aligned}
 \theta = & S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge \\
 & (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge \\
 & (S_{3,2} \leftrightarrow (S_{2,1} \wedge I_{1,a})) \wedge (S_{3,3} \leftrightarrow (S_{2,2} \wedge I_{2,a})) \wedge (S_{3,4} \leftrightarrow (S_{2,3} \wedge I_{3,a})) \wedge \\
 & (S_{4,2} \leftrightarrow (S_{3,1} \wedge I_{1,a})) \wedge (S_{4,3} \leftrightarrow (S_{3,2} \wedge I_{2,a})) \wedge (S_{4,4} \leftrightarrow (S_{3,3} \wedge I_{3,a})) \wedge \\
 & (S_{5,2} \leftrightarrow ((S_{3,1} \vee S_{4,1}) \wedge I_{1,n})) \wedge (S_{5,3} \leftrightarrow ((S_{3,2} \vee S_{4,2}) \wedge I_{2,n})) \wedge (S_{5,4} \leftrightarrow ((S_{3,3} \vee S_{4,3}) \wedge I_{3,n})) \wedge \\
 & (S_{6,2} \leftrightarrow (S_{5,1} \wedge I_{1,a})) \wedge (S_{6,3} \leftrightarrow (S_{5,2} \wedge I_{2,a})) \wedge (S_{6,4} \leftrightarrow (S_{5,3} \wedge I_{3,a})) \wedge \\
 & (S_{3,4} \vee S_{4,4} \vee S_{6,4})
 \end{aligned}$$

THE CONVERSION

Input: mrf



We are done!

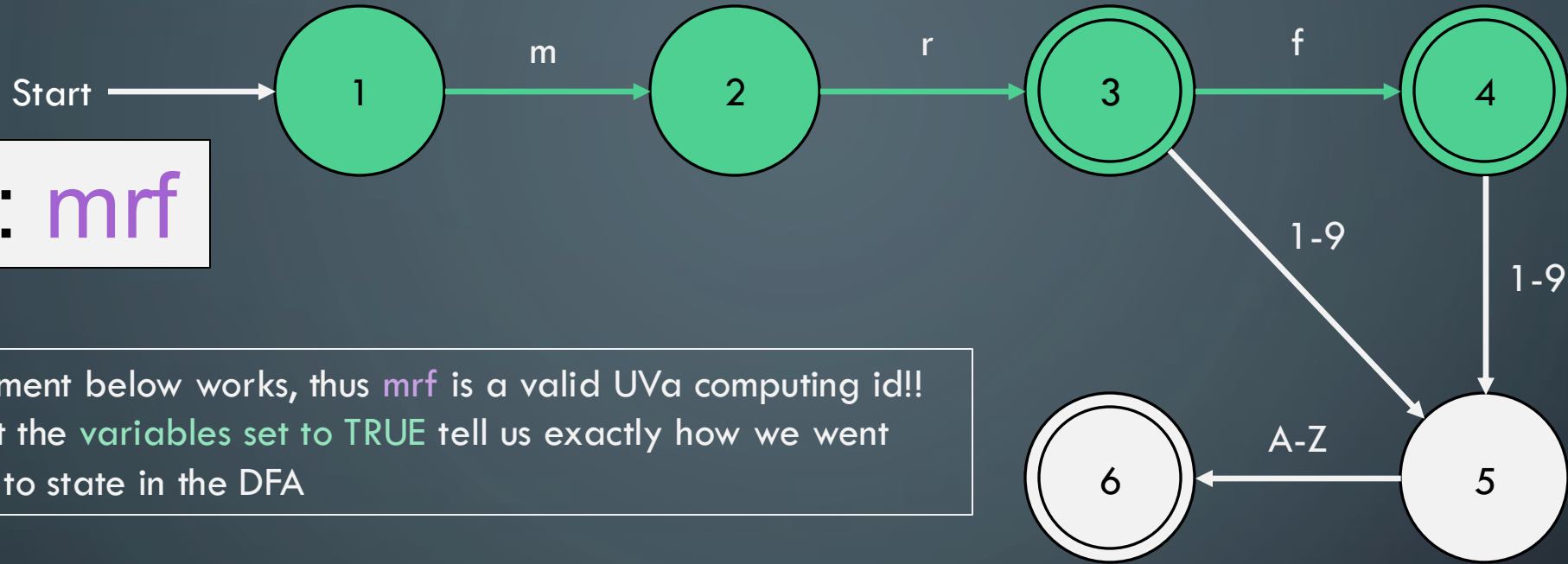
Claim: We have two different ways to test if mrf is valid computing id. We can run DFA or we can try to satisfy the equation below!

$$\begin{aligned} \theta = & S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge \\ & (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge \\ & (S_{3,2} \leftrightarrow (S_{2,1} \wedge I_{1,a})) \wedge (S_{3,3} \leftrightarrow (S_{2,2} \wedge I_{2,a})) \wedge (S_{3,4} \leftrightarrow (S_{2,3} \wedge I_{3,a})) \wedge \\ & (S_{4,2} \leftrightarrow (S_{3,1} \wedge I_{1,a})) \wedge (S_{4,3} \leftrightarrow (S_{3,2} \wedge I_{2,a})) \wedge (S_{4,4} \leftrightarrow (S_{3,3} \wedge I_{3,a})) \wedge \\ & (S_{5,2} \leftrightarrow ((S_{3,1} \vee S_{4,1}) \wedge I_{1,n})) \wedge (S_{5,3} \leftrightarrow ((S_{3,2} \vee S_{4,2}) \wedge I_{2,n})) \wedge (S_{5,4} \leftrightarrow ((S_{3,3} \vee S_{4,3}) \wedge I_{3,n})) \wedge \\ & (S_{6,2} \leftrightarrow (S_{5,1} \wedge I_{1,a})) \wedge (S_{6,3} \leftrightarrow (S_{5,2} \wedge I_{2,a})) \wedge (S_{6,4} \leftrightarrow (S_{5,3} \wedge I_{3,a})) \wedge \\ & (S_{3,4} \vee S_{4,4} \vee S_{6,4}) \end{aligned}$$

THE CONVERSION

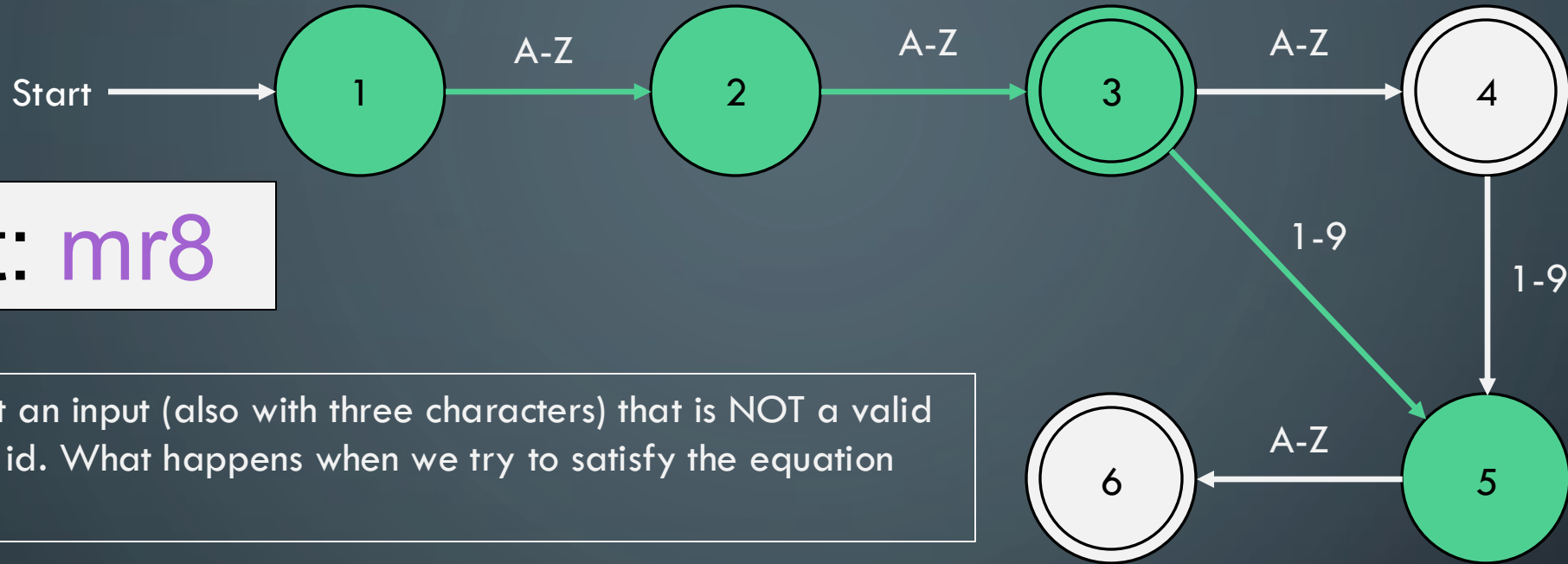
Input: **mrf**

The assignment below works, thus **mrf** is a valid UVa computing id!!
Notice that the **variables set to TRUE** tell us exactly how we went from state to state in the DFA



$$\begin{aligned}
 \theta = & S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (I_{3,a} \wedge \neg I_{3,n}) \wedge \\
 & (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge \\
 & (S_{3,2} \leftrightarrow (S_{2,1} \wedge I_{1,a})) \wedge (S_{3,3} \leftrightarrow (S_{2,2} \wedge I_{2,a})) \wedge (S_{3,4} \leftrightarrow (S_{2,3} \wedge I_{3,a})) \wedge \\
 & (S_{4,2} \leftrightarrow (S_{3,1} \wedge I_{1,a})) \wedge (S_{4,3} \leftrightarrow (S_{3,2} \wedge I_{2,a})) \wedge (S_{4,4} \leftrightarrow (S_{3,3} \wedge I_{3,a})) \wedge \\
 & (S_{5,2} \leftrightarrow ((S_{3,1} \vee S_{4,1}) \wedge I_{1,n})) \wedge (S_{5,3} \leftrightarrow ((S_{3,2} \vee S_{4,2}) \wedge I_{2,n})) \wedge (S_{5,4} \leftrightarrow ((S_{3,3} \vee S_{4,3}) \wedge I_{3,n})) \wedge \\
 & (S_{6,2} \leftrightarrow (S_{5,1} \wedge I_{1,a})) \wedge (S_{6,3} \leftrightarrow (S_{5,2} \wedge I_{2,a})) \wedge (S_{6,4} \leftrightarrow (S_{5,3} \wedge I_{3,a})) \wedge \\
 & (S_{3,4} \vee S_{4,4} \vee S_{6,4})
 \end{aligned}$$

THE CONVERSION

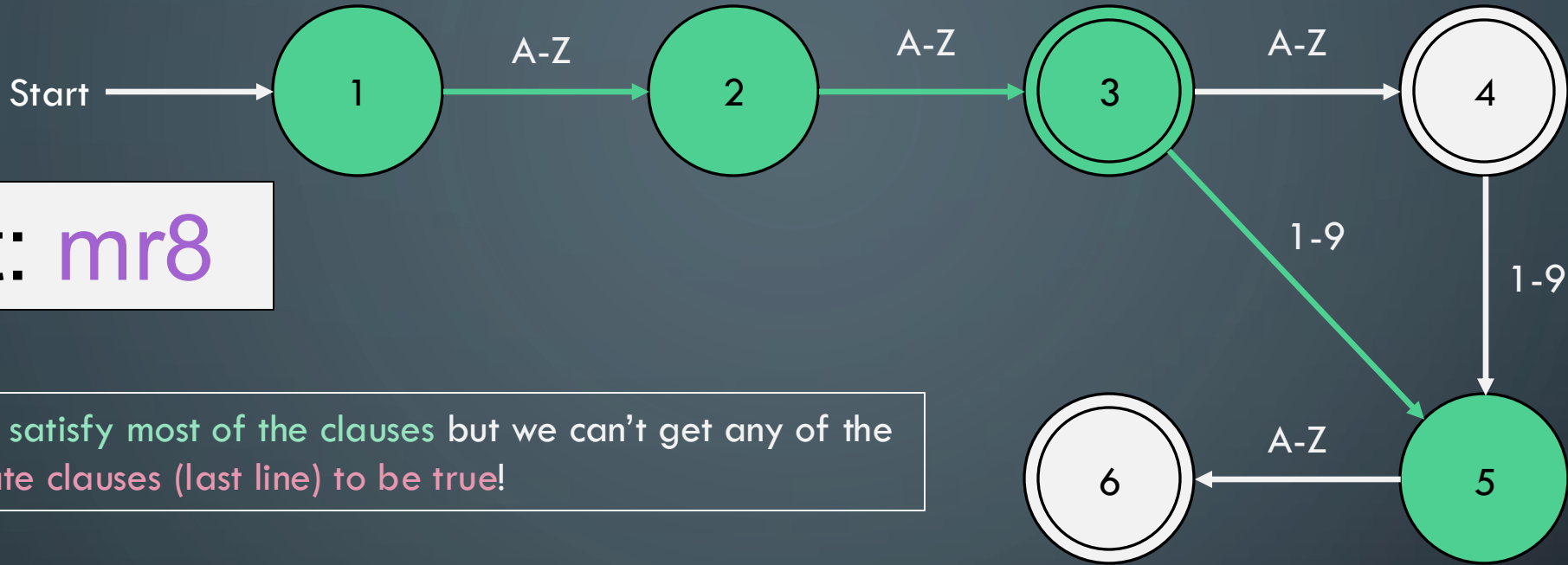


Input: mr8

How about an input (also with three characters) that is NOT a valid computing id. What happens when we try to satisfy the equation below?

$$\begin{aligned} \theta = & S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (\neg I_{3,a} \wedge I_{3,n}) \wedge \\ & (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge \\ & (S_{3,2} \leftrightarrow (S_{2,1} \wedge I_{1,a})) \wedge (S_{3,3} \leftrightarrow (S_{2,2} \wedge I_{2,a})) \wedge (S_{3,4} \leftrightarrow (S_{2,3} \wedge I_{3,a})) \wedge \\ & (S_{4,2} \leftrightarrow (S_{3,1} \wedge I_{1,a})) \wedge (S_{4,3} \leftrightarrow (S_{3,2} \wedge I_{2,a})) \wedge (S_{4,4} \leftrightarrow (S_{3,3} \wedge I_{3,a})) \wedge \\ & (S_{5,2} \leftrightarrow ((S_{3,1} \vee S_{4,1}) \wedge I_{1,n})) \wedge (S_{5,3} \leftrightarrow ((S_{3,2} \vee S_{4,2}) \wedge I_{2,n})) \wedge (S_{5,4} \leftrightarrow ((S_{3,3} \vee S_{4,3}) \wedge I_{3,n})) \wedge \\ & (S_{6,2} \leftrightarrow (S_{5,1} \wedge I_{1,a})) \wedge (S_{6,3} \leftrightarrow (S_{5,2} \wedge I_{2,a})) \wedge (S_{6,4} \leftrightarrow (S_{5,3} \wedge I_{3,a})) \wedge \\ & (S_{3,4} \vee S_{4,4} \vee S_{6,4}) \end{aligned}$$

THE CONVERSION

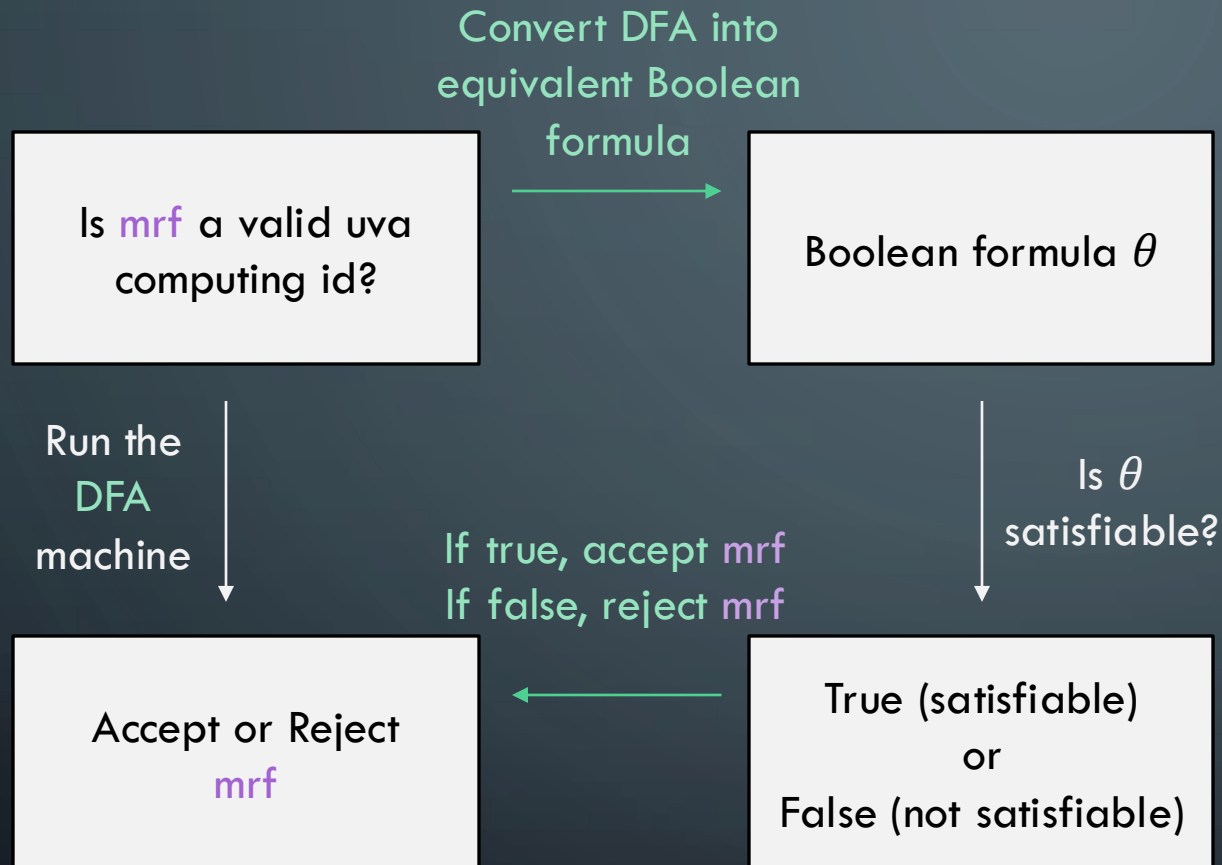


Input: **mr8**

We can satisfy most of the clauses but we can't get any of the final state clauses (last line) to be true!

$$\begin{aligned}
 \theta = & S_{1,1} \wedge (I_{1,a} \wedge \neg I_{1,n}) \wedge (I_{2,a} \wedge \neg I_{2,n}) \wedge (\neg I_{3,a} \wedge I_{3,n}) \wedge \\
 & (S_{2,2} \leftrightarrow (S_{1,1} \wedge I_{1,a})) \wedge (S_{2,3} \leftrightarrow (S_{1,2} \wedge I_{2,a})) \wedge (S_{2,4} \leftrightarrow (S_{1,3} \wedge I_{3,a})) \wedge \\
 & (S_{3,2} \leftrightarrow (S_{2,1} \wedge I_{1,a})) \wedge (S_{3,3} \leftrightarrow (S_{2,2} \wedge I_{2,a})) \wedge (S_{3,4} \leftrightarrow (S_{2,3} \wedge I_{3,a})) \wedge \\
 & (S_{4,2} \leftrightarrow (S_{3,1} \wedge I_{1,a})) \wedge (S_{4,3} \leftrightarrow (S_{3,2} \wedge I_{2,a})) \wedge (S_{4,4} \leftrightarrow (S_{3,3} \wedge I_{3,a})) \wedge \\
 & (S_{5,2} \leftrightarrow ((S_{3,1} \vee S_{4,1}) \wedge I_{1,n})) \wedge (S_{5,3} \leftrightarrow ((S_{3,2} \vee S_{4,2}) \wedge I_{2,n})) \wedge (S_{5,4} \leftrightarrow ((S_{3,3} \vee S_{4,3}) \wedge I_{3,n})) \wedge \\
 & (S_{6,2} \leftrightarrow (S_{5,1} \wedge I_{1,a})) \wedge (S_{6,3} \leftrightarrow (S_{5,2} \wedge I_{2,a})) \wedge (S_{6,4} \leftrightarrow (S_{5,3} \wedge I_{3,a})) \wedge \\
 & (S_{3,4} \vee S_{4,4} \vee S_{6,4})
 \end{aligned}$$

REDUCTION IDEA



Reduction Idea:

Gives us TWO different ways to “run” this program. We can run the DFA directly OR we can convert into a formula and solve the Boolean formula instead.

OTHER COMPLEXITY CLASSES

COMPLEXITY CLASSES

